

Fitting the Hooker et al. (2020) model to both sexes

John D. Robinson

2026-09-08

Contents

Fitting a model similar to that from Hooker et al. (2020) with <i>both sexes</i>	1
Load the data	6
Rerun <code>fit.R</code>	7
Fit the model	17
Prepare Data	17
Posterior Samples	34
Activity Center Density	58
Save the workspace	61

Fitting a model similar to that from Hooker et al. (2020) with *both sexes*

The manuscript by [Hooker et al. \(2020\)](#) fit an open-population SCR model to recapture data from female black bears in central GA using hair samples collected over a period of 5 years (2012-2016). They compared several models (see their Table 2), but the best-fit model included a constant survival rate across years, density-dependent per capita recruitment, year-specific baseline detection probabilities, and a local behavioral response to capture. Estimates of abundance from their analysis ranged from 106 to 166 across the study period.

Here, we modify code from Hooker et al. (2020) (available at: github.com/rbchan/ga-bear-pva) to fit and compare a very similar model to capture histories from hair samples collected in 2023 and 2024 (from *both male and female bears*) in the same area. In particular, the analysis below draws heavily from Hooker et al.'s `fit.R` script and their [JAGS model](#), and uses their [state space .tif](#) file.

The [JAGS](#) model from the Hooker manuscript is modified below. Specifically, we remove the density-dependence of per-capita recruitment (with only 1 recruitment period, this should have little effect on the model). We also fit the model to data from both sexes and include several sex-specific parameters.

Differences between the sexes were included for the following parameters:

- i. p_0 - baseline detection probability, probability of detection when an individual's activity center is in the same location as a hair snare

- ii. κ - the (local) behavioral effect of capture, assumed to increase p_0 for individuals that have previously been detected in a specific location (*i.e.*, “trap happy” behavior)
- iii. σ - the decay rate of detection probability with distance (related to movement)
- iv. ϕ - (apparent) survival between primary sampling periods (years)
- v. ψ - the data augmentation parameter, as males and females are augmented separately in the model

We augment the male and female capture histories separately, so much of the code in the model is duplicated. This also means we have separate objects in the model for the male and female activity centers, individual alive/dead/available statuses, and so on. We will need to initialize these variables separately for males and females, so there will be a bit more to organize in the JAGS `data` and `inits` portions.

Here’s the JAGS model we’re using.

```
writeLines(readLines("../input/CGA_SCR.jag"))

## model {
##
##   ## Priors
##   # for sex-specific parameters
##   for(sx in 1:2) {
##     kappa[sx] ~ dbeta(1,1) #behavioral effect of capture
##     phi[sx] ~ dbeta(1,1) #survival
##     sigma[sx] ~ dgamma(0.01, 0.01) #decay of det. prob. with dist.
##
##     #year- and sex-specific parameters
##     for(t in 1:nYears) {
##       #baseline det. prob.
##       p0st[sx,t,2] ~ dbeta(1,1) #previously captured individuals
##       p0st[sx,t,1] <- p0st[sx,t,2]*kappa[sx] #naive individuals
##     }
##     psi[sx,1] ~ dbeta(1,1) #data augmentation - year 1
##   }
##
##   gamma ~ dunif(0,1) #recruitment (each sex, same rate)
##
##   #Recruitment model and expected abundances
##   ENf[1] <- Mf*psi[1,1]
##   ENm[1] <- Mm*psi[2,1]
##   EN[1] <- ENf[1]+ENm[1]
##   for(t in 2:nYears) {
##     #expected male and female recruits
##     ERf[t] <- Nf[t-1]*gamma
##     ERm[t] <- Nm[t-1]*gamma
##     ER[t] <- ERf[t]+ERm[t]
##
##     #data augmentation for future years
##     #females
##     psi[1,t] <- ERf[t]/sum(aF[,t-1])
##     psi[2,t] <- ERm[t]/sum(aM[,t-1])
##
##     #expected abundances
##     ENf[t] <- Nf[t-1]*phi[1] + ERf[t]
##     ENm[t] <- Nm[t-1]*phi[2] + ERm[t]
##     EN[t] <- ENf[t]+ENm[t]
```

```

## }
##
## ## State process
## # activity center & alive / dead / available for FEMALES
## for(i in 1:Mf) {
##     #Activity center (no movement)
##     sF[i,1] ~ dunif(xlim[1], xlim[2])
##     sF[i,2] ~ dunif(ylim[1], ylim[2])
##     #ones trick for females
##     onesF[i] ~ dbern(S[round((ylim[2]-sF[i,2])/delta + 0.5),
##                       round((sF[i,1]-xlim[1])/delta + 0.5)])
##
##     #Initial statuses
##     zF[i,1] ~ dbern(psi[1,1])
##     aF[i,1] <- 1-zF[i,1]
##
##     #Future statuses
##     for(t in 2:nYears) {
##         EzF[i,t] <- zF[i,t-1]*phi[1] + aF[i,t-1]*psi[1,t]
##         zF[i,t] ~ dbern(EzF[i,t])
##         aF[i,t] <- aF[i,t-1]*(1-zF[i,t])
##     }
##     everAliveF[i] <- max(zF[i,])
## }
##
## #activity center & alive / dead / available for MALES
## for(i in 1:Mm) {
##     #Activity center (no movement)
##     sM[i,1] ~ dunif(xlim[1], xlim[2])
##     sM[i,2] ~ dunif(ylim[1], ylim[2])
##     #ones trick for males
##     onesM[i] ~ dbern(S[round((ylim[2]-sM[i,2])/delta + 0.5),
##                       round((sM[i,1]-xlim[1])/delta + 0.5)])
##
##     #Initial statuses
##     zM[i,1] ~ dbern(psi[2,1])
##     aM[i,1] <- 1-zM[i,1]
##
##     #Future statuses
##     for(t in 2:nYears) {
##         EzM[i,t] <- zM[i,t-1]*phi[2] + aM[i,t-1]*psi[2,t]
##         zM[i,t] ~ dbern(EzM[i,t])
##         aM[i,t] <- aM[i,t-1]*(1-zM[i,t])
##     }
##     everAliveM[i] <- max(zM[i,])
## }
##
## ## Females
## # Observed data
## for(i in 1:nFem) {
##     for(t in 1:nYears) {
##         for(j in oper.traps[1:nTrapsYr[t],t]) {
##             dF[i,j,t] <- sqrt((sF[i,1]-x[j,1])^2 +
##                               (sF[i,2]-x[j,2])^2)
##         }
##     }
## }

```

```

##           for(k in 1:nWeeks) {
##               #Expected capture
##               EyF[i,j,k,t] <- p0st[1,t,prevcapF[i,j,k,t]]*
##                   exp(-dF[i,j,t]^2/(2*sigma[1]^2))*
##                   zF[i,t]
##               #Capture
##               yF[i,j,k,t] ~ dbern(EyF[i,j,k,t])
##           }
##       }
##   }
##
##   # Augmented data
##   for(i in (nFem+1):Mf) {
##       for(t in 1:nYears) {
##           for(j in oper.traps[1:nTrapsYr[t],t]) {
##               dF[i,j,t] <- sqrt((sF[i,1]-x[j,1])^2 +
##                               (sF[i,2]-x[j,2])^2)
##               for(k in 1:nWeeks) {
##                   #Augmented individuals have no previous capture
##                   EyF[i,j,k,t] <- p0st[1,t,1]*
##                       exp(-dF[i,j,t]^2/(2*sigma[1]^2))*
##                       zF[i,t]
##               }
##               #Prob of no captures of i across nWeeks
##               prNoDetWeekF[i,j,t] <- prod(1-EyF[i,j,,t])
##           }
##           #Prob of capturing i at least once in year t
##           prDetF[i,t] <- 1-prod(prNoDetWeekF[i,oper.traps[1:nTrapsYr[t],t],t])
##           yF.zero[i,t] ~ dbern(prDetF[i,t])
##       }
##   }
##
##   ## Males
##   # Observed data
##   for(i in 1:nMale) {
##       for(t in 1:nYears) {
##           for(j in oper.traps[1:nTrapsYr[t],t]) {
##               dM[i,j,t] <- sqrt((sM[i,1]-x[j,1])^2 +
##                               (sM[i,2]-x[j,2])^2)
##               for(k in 1:nWeeks) {
##                   #Expected capture
##                   EyM[i,j,k,t] <- p0st[2,t,prevcapM[i,j,k,t]]*
##                       exp(-dM[i,j,t]^2/(2*sigma[2]^2))*
##                       zM[i,t]
##                   #Capture
##                   yM[i,j,k,t] ~ dbern(EyM[i,j,k,t])
##               }
##           }
##       }
##   }
##
##   # Augmented data
##   for(i in (nMale+1):Mm) {

```

```

##      for(t in 1:nYears) {
##          for(j in oper.traps[1:nTrapsYr[t],t]) {
##              dM[i,j,t] <- sqrt((sM[i,1]-x[j,1])^2 +
##                  (sM[i,2]-x[j,2])^2)
##              for(k in 1:nWeeks) {
##                  #Augmented individuals have no previous capture
##                  EyM[i,j,k,t] <- p0st[2,t,1]*
##                      exp(-dM[i,j,t]^2/(2*sigma[2]^2))*
##                      zM[i,t]
##              }
##              #Prob of no captures of i across nWeeks
##              prNoDetWeekM[i,j,t] <- prod(1-EyM[i,j,,t])
##          }
##          #Prob of capturing i at least once in year t
##          prDetM[i,t] <- 1-prod(prNoDetWeekM[i,oper.traps[1:nTrapsYr[t],t],t))
##          yM.zero[i,t] ~ dbern(prDetM[i,t])
##      }
##  }
##
##  #Derived parameters
##  totF <- sum(everAliveF)
##  totM <- sum(everAliveM)
##  nTotal <- totF+totM
##  for(t in 1:nYears) {
##      #Abundances
##      Nf[t] <- sum(zF[,t])
##      Nm[t] <- sum(zM[,t])
##      N[t] <- Nf[t]+Nm[t]
##      #Densities
##      #expected
##      ED[t] <- EN[t]/poly.area
##      EDf[t] <- ENf[t]/poly.area
##      EDm[t] <- ENm[t]/poly.area
##      #realized
##      RD[t] <- N[t]/poly.area
##      RDf[t] <- Nf[t]/poly.area
##      RDm[t] <- Nm[t]/poly.area
##  }
##
##  for(t in 2:nYears) {
##      survivingF <- zF[,t-1] %*% zF[,t]
##      survivingM <- zM[,t-1] %*% zM[,t]
##      recruitF <- (1-zF[,t-1]) %*% zF[,t]
##      recruitM <- (1-zM[,t-1]) %*% zM[,t]
##      lambdaF <- Nf[t]/max(Nf[t-1], 0.1)
##      lambdaM <- Nm[t]/max(Nm[t-1], 0.1)
##      lambda <- N[t]/max(N[t-1], 0.1)
##  }
##
##  }

```

Load the data

We do NOT need to create a combined capture history object, since we augment male and female capture histories separately in the model.

```
#Load and collect the necessary data objects
```

```
load("CGA_CH.Rda")
```

```
load("../sampleinfo/snareLocations.Rda")
```

```
ls()
```

```
## [1] "F23" "F24" "Fch" "locs" "M23" "M24" "Mch" "oper"
```

```
#Some summary stats
```

```
source("../input/sumstats_4d.R")
```

```
#capture history lists have 3 elements:
```

```
# y - the capture history, 0 for no detection, 1 for detection
```

```
# pc - previous capture, has individual i been seen in trap j before?
```

```
# clone - clone indices (finalClone) that correspond to rows of y and pc
```

```
names(Mch)
```

```
## [1] "y" "pc" "clone"
```

```
names(Fch)
```

```
## [1] "y" "pc" "clone"
```

```
#Check that j, k, and t dimensions of the y and pc arrays in Mch and Fch are =
```

```
dim(Mch$y)
```

```
## [1] 77 154 7 2
```

```
dim(Fch$y)
```

```
## [1] 88 154 7 2
```

```
dim(Mch$pc)
```

```
## [1] 77 154 7 2
```

```
dim(Fch$pc)
```

```
## [1] 88 154 7 2
```

```
#Overall summaries of captures
```

```
sumstats_4d(Fch$y)
```

```
##      ss.totN      ss.totcaps      ss.avgcaps ss.indwsprecap      ss.avgtraps
##      88.000000      260.000000      2.954545      31.000000      1.602273
##      ss.avgyrs
##      1.284091
```

```
sumstats_4d(Mch$y)
```

```
##      ss.totN      ss.totcaps      ss.avgcaps ss.indwsprecap      ss.avgtraps
##      77.000000      254.000000        3.298701        26.000000        2.246753
##      ss.avgyrs
##      1.194805
```

```
#Yearly summaries of captures
```

```
sumstats_3d(F23$y)
```

```
##      ss.totN      ss.totcaps      ss.avgcaps ss.indwsprecap      ss.avgtraps
##      43.000000      109.000000        2.534884        13.000000        1.511628
```

```
sumstats_3d(F24$y)
```

```
##      ss.totN      ss.totcaps      ss.avgcaps ss.indwsprecap      ss.avgtraps
##      70.000000      151.000000        2.157143        19.000000        1.400000
```

```
sumstats_3d(M23$y)
```

```
##      ss.totN      ss.totcaps      ss.avgcaps ss.indwsprecap      ss.avgtraps
##      55.000000      116.000000        2.109091        21.000000        1.690909
```

```
sumstats_3d(M24$y)
```

```
##      ss.totN      ss.totcaps      ss.avgcaps ss.indwsprecap      ss.avgtraps
##      37.000000      138.000000        3.729730        13.000000        2.540541
```

Rerun fit.R

From here on out, we'll follow along with the [fit.R](#) script from Hooker et al. (2020). Note that there will be a number of modifications from fit.R in the following chunks. There are several intermediate sample collections in that script that are not implemented here (*e.g.*, for forecasting). However, we do use the same basic code to fit the model.

```
## Fit open population SCR models to central GA black bear data
```

```
library(rjags)
```

```
## Loading required package: coda
```

```
## Linked to JAGS 4.3.2
```

```
## Loaded modules: basemod,bugs
```

```
library(lattice)
```

```
library(raster)
```

```
## Loading required package: sp
```

```
# For consistency, rename the objects to match their scheme...
ch4d.f <- Fch$y
prevcap.f <- Fch$pc
ch4d.m <- Mch$y
prevcap.m <- Mch$pc
traps <- locs
```

```
## Inspect the loaded data objects
str(ch4d.f)
```

```
## num [1:88, 1:154, 1:7, 1:2] 0 0 0 0 0 0 0 0 0 0 ...
```

```
str(ch4d.m)
```

```
## num [1:77, 1:154, 1:7, 1:2] 0 0 0 0 0 0 0 0 0 0 ...
```

```
str(traps)
```

```
## 'data.frame': 154 obs. of 2 variables:
## $ East : num 248187 255814 248920 250904 252875 ...
## $ North: num 3586614 3587061 3588050 3587890 3588670 ...
```

```
## Check that traps are in the same order in both objects
## Must be TRUE
all(rownames(traps)==dimnames(ch4d.f)[[3]])
```

```
## [1] TRUE
```

```
all(rownames(traps)==dimnames(ch4d.m)[[3]])
```

```
## [1] TRUE
```

```
all(rownames(oper)==rownames(traps))
```

```
## [1] TRUE
```

```
## Format data
```

```
yF <- ch4d.f ## 4D encounter histories
str(yF)
```

```
## num [1:88, 1:154, 1:7, 1:2] 0 0 0 0 0 0 0 0 0 0 ...
```

```
yM <- ch4d.m
str(yM)
```

```
## num [1:77, 1:154, 1:7, 1:2] 0 0 0 0 0 0 0 0 0 0 ...
```

```

y.dim <- dim(yF)
nFem <- dim(yF)[1]
nMale <- dim(yM)[1]
nTraps <- dim(yF)[2]
nWeeks <- dim(yF)[3]
nYears <- dim(yF)[4]

## Females
Mf <- 500 # Data aug. Maximum # offemales that could have been alive
all(rownames(prevcap.f)==rownames(yF)) ## Must be TRUE

## [1] TRUE

## Non-spatial, robust design format
yF.nonsp.robust <- apply(yF>0, c(1,3,4), any)*1L
str(yF.nonsp.robust)

## int [1:88, 1:7, 1:2] 0 0 0 0 0 0 0 0 0 0 ...

## Non-spatial, non-robust format
yF.nonsp.nonrobust <- apply(yF.nonsp.robust>0,
                           c(1,3), any)*1L
str(yF.nonsp.nonrobust)

## int [1:88, 1:2] 1 1 1 1 0 1 1 1 1 1 ...

## Spatial, non-robust format
yF.sp.nonrobust <- apply(yF>0, c(1,2,4), any)*1L
str(yF.sp.nonrobust)

## int [1:88, 1:154, 1:2] 0 0 0 0 0 0 0 0 1 0 0 ...

## First and last detections
first.detF <- apply(yF.nonsp.nonrobust>0, 1, function(x) min(which(x)))
last.detF <- apply(yF.nonsp.nonrobust>0, 1, function(x) max(which(x)))
known.aliveF <- last.detF-first.detF+1
table(known.aliveF)

## known.aliveF
## 1 2
## 63 25

## Data augmentation
yFaug <- array(0, c(Mf, nTraps, nWeeks, nYears))
yFaug[1:nFem,,] <- yF

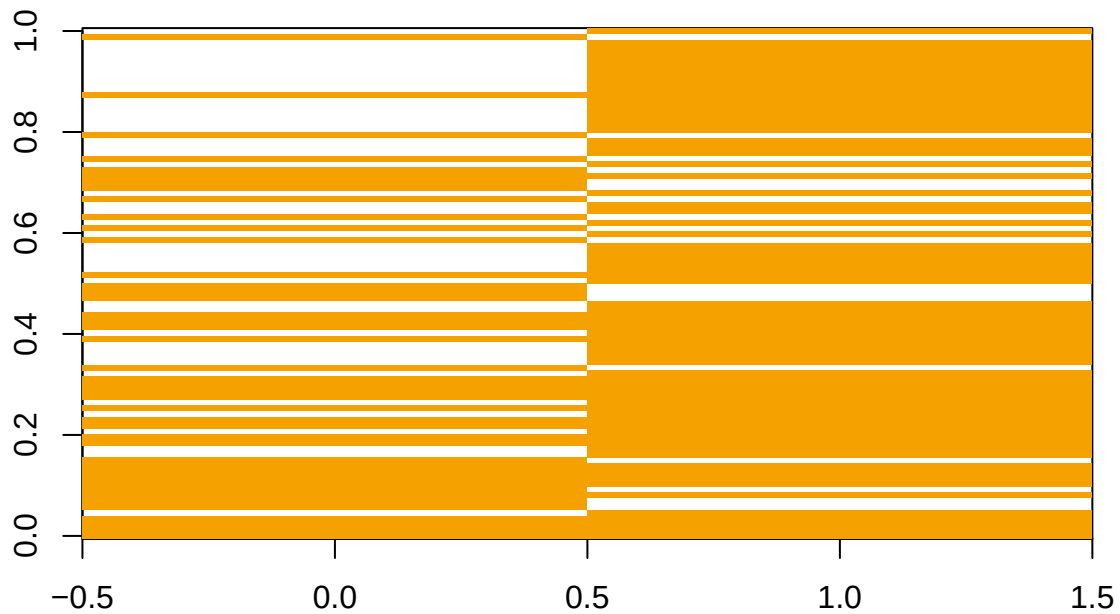
str(yFaug)

## num [1:500, 1:154, 1:7, 1:2] 0 0 0 0 0 0 0 0 0 0 ...

```

```
## Known values of the alive/dead state matrix
zFdata.aug <- matrix(NA, Mf, nYears)
for(i in 1:nFem) {
  zFdata.aug[i,first.detF[i]:last.detF[i]] <- 1
}

## Visualize the observed information about lifetime
image(t(zFdata.aug[1:nFem,]))
```



```
## Bears known to be alive in each year
colSums(zFdata.aug[1:nFem,], na.rm=TRUE)
```

```
## [1] 43 70
```

```
## Males
Mm <- 500 # Data aug. Maximum # of males that could have been alive
all(rownames(prevcap.m)==rownames(yM)) ## Must be TRUE
```

```
## [1] TRUE
```

```
## Non-spatial, robust design format
yM.nonsp.robust <- apply(yM>0, c(1,3,4), any)*1L
str(yM.nonsp.robust)
```

```
## int [1:77, 1:7, 1:2] 0 0 0 0 1 0 0 0 0 0 ...
```

```
## Non-spatial, non-robust format
yM.nonsp.nonrobust <- apply(yM.nonsp.robust>0,
                           c(1,3), any)*1L
str(yM.nonsp.nonrobust)
```

```
## int [1:77, 1:2] 0 1 1 1 1 1 1 1 1 1 ...
```

```
## Spatial, non-robust format
yM.sp.nonrobust <- apply(yM>0, c(1,2,4), any)*1L
str(yM.sp.nonrobust)
```

```
## int [1:77, 1:154, 1:2] 0 0 0 0 0 0 0 0 0 0 ...
```

```
## First and last detections
first.detM <- apply(yM.nonsp.nonrobust>0, 1, function(x) min(which(x)))
last.detM <- apply(yM.nonsp.nonrobust>0, 1, function(x) max(which(x)))
known.aliveM <- last.detM-first.detM+1
table(known.aliveM)
```

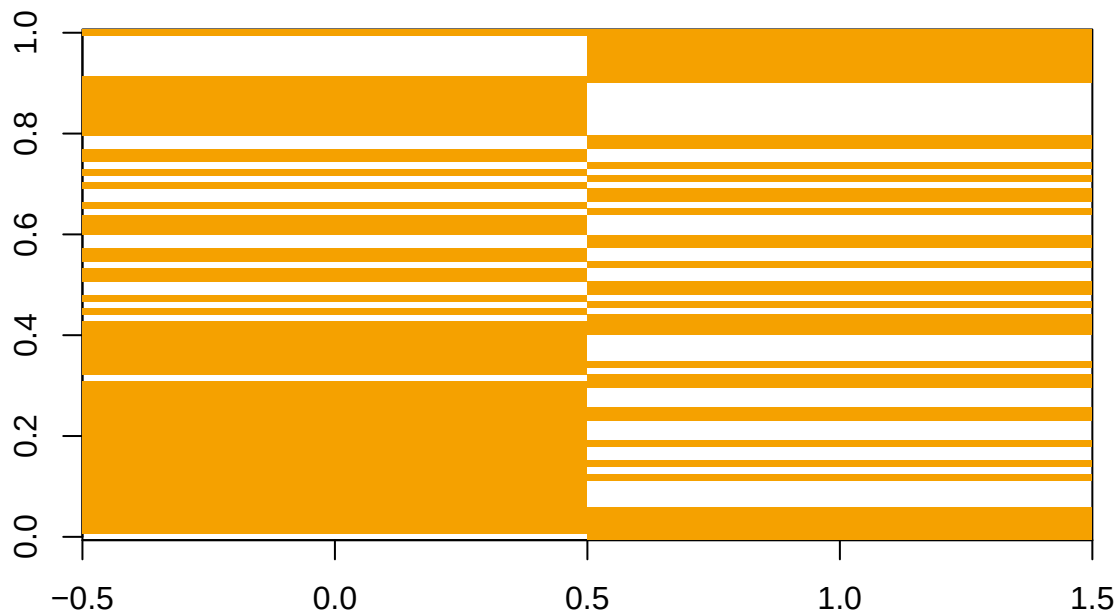
```
## known.aliveM
## 1 2
## 62 15
```

```
## Data augmentation
yMaug <- array(0, c(Mm, nTraps, nWeeks, nYears))
yMaug[1:nMale,,] <- yM
str(yMaug)
```

```
## num [1:500, 1:154, 1:7, 1:2] 0 0 0 0 0 0 0 0 0 0 ...
```

```
## Known values of the alive/dead state matrix
zMdata.aug <- matrix(NA, Mm, nYears)
for(i in 1:nMale) {
  zMdata.aug[i,first.detM[i]:last.detM[i]] <- 1
}
```

```
## Visualize the observed information about lifetime
image(t(zMdata.aug[1:nMale,]))
```



```
## Bears known to be alive in each year
colSums(zMdata.aug[1:nMale,], na.rm=TRUE)
```

```
## [1] 55 37
```

```
## Operational status of traps
nTrapsYr <- colSums(oper)
nTrapsYr
```

```
## Yr2023 Yr2024
##    130    138
```

```
oper.traps <- matrix(0L, nTraps, nYears)
for(i in 1:nYears) {
  oper.traps[1:nTrapsYr[i],i] <- which(oper[,i]==1)
}
```

```
## State-space
x.range <- range(traps[,1])
y.range <- range(traps[,2])
buffer <- 4000
xlim0 <- x.range+c(-buffer, buffer)
ylim0 <- y.range+c(-buffer, buffer)
```

```

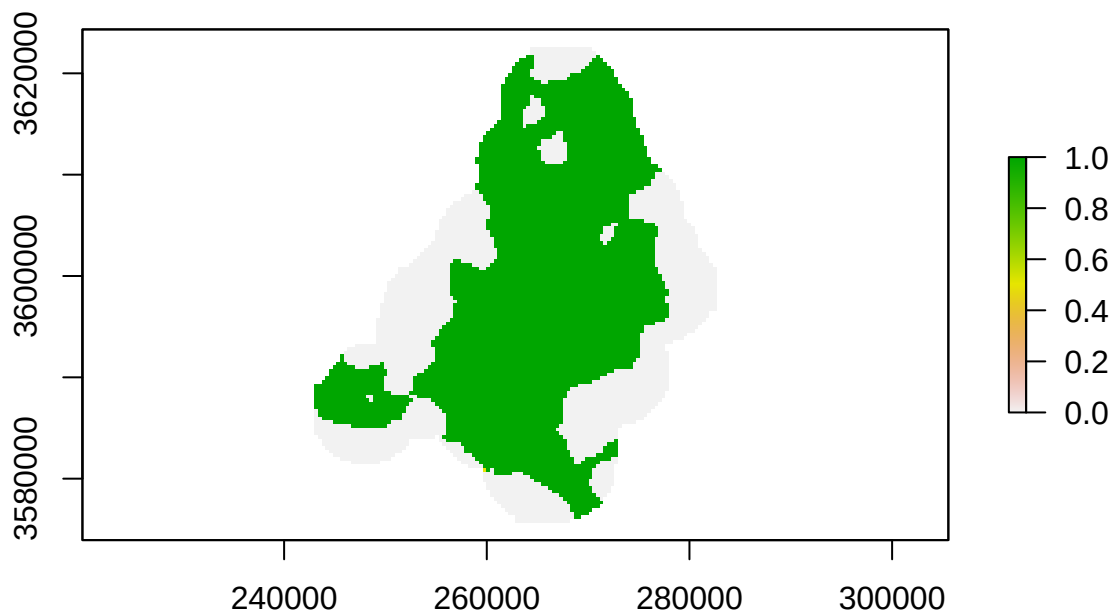
## Need to point to the location where I've saved this file
S360 <- raster("../input/state-space360.tif")

#Convert to UTM-17N-WGS84 to match snareLocations.Rda
newproj <- "+proj=utm +zone=17 +datum=WGS84 +units=m +no_defs +type=crs"
S360 <- projectRaster(S360, crs=newproj)

traps.in <- round(extract(S360, traps))

plot(S360)

```



```

## Convert raster to a data.frame
S360dat <- as.data.frame(S360, xy=TRUE)
names(S360dat) <- c("x", "y", "S360")
S360dat$x <- floor(S360dat$x)
S360dat$y <- floor(S360dat$y)
table(diff(S360dat$x))

```

```

##
## -42840    360
##    139  16660

```

```

table(diff(S360dat$y))

##
##   -360      0
##   139 16660

nPix360 <- nrow(S360dat)
dist360 <- matrix(NA, nPix360, nTraps)
for(j in 1:nTraps) {
  dist360[,j] <- sqrt((S360dat$x-traps[j,1])^2 +
                     (S360dat$y-traps[j,2])^2)
}

S360dat$pixIn <- (rowSums(dist360<buffer)>0)*1L
table(S360dat$pixIn)

##
##      0      1
## 10267  6533

dim.S360 <- dim(S360)

S360dat$habitat <- ifelse((S360dat$S360>0) &
                        (S360dat$pixIn>0),
                        1L, 0L)

```

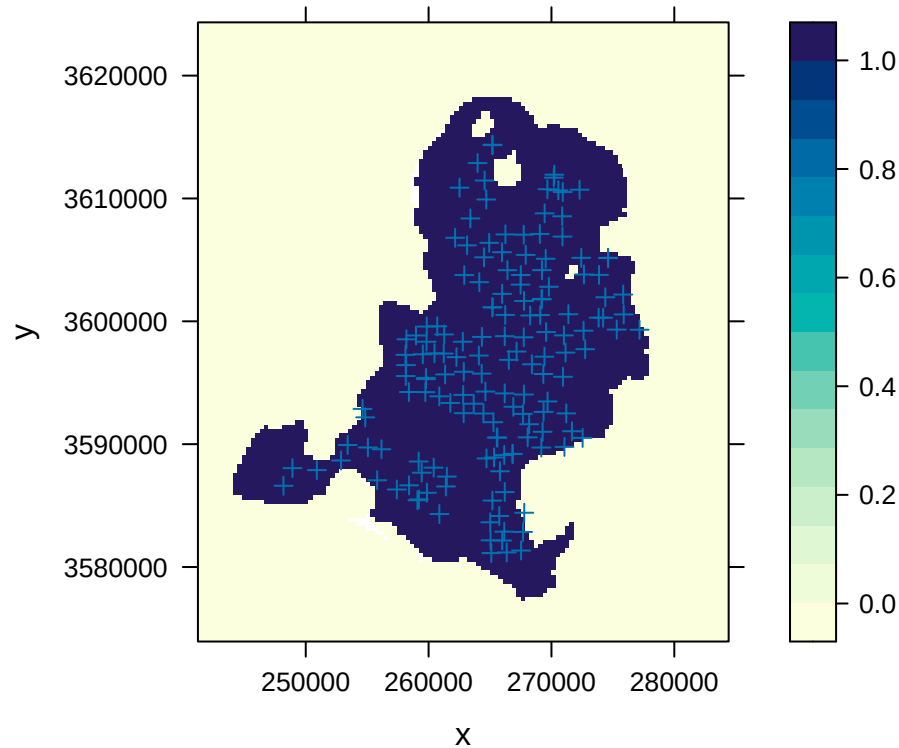
Before proceeding further, it's important to note that the trap locations used in this study were not identical to those from Hooker et al. (2020). So it won't be surprising if we run into some issues where our traps are located outside of their state space. With that said, let's take a look at how our traps align.

Sure enough, there were some slight disagreements here. The **S** object ended up with some NA entries that led to errors in the **JAGS** run. Replacing those with 0 or 1 seems to get around the issue. It seems that these are cells that were not considered in the Hooker et al. (2020) study, so we don't have any information on habitat characteristics. With that in mind, it's more conservative to assume that they are NOT habitat, than to allow bear activity centers to be located in these cells.

```

## Visualize traps and state space
levelplot(habitat ~ x+y, S360dat, aspect='iso',
  panel=function(...) {
    panel.levelplot(...)
    lpoints(traps, pch=3)
  })

```



```
delta <- res(S360)[1]
xseq <- sort(unique(S360dat$x))
yseq <- sort(unique(S360dat$y), decreasing=TRUE)
xlim <- range(xseq)+c(-delta/2,delta/2)
ylim <- range(yseq)+c(-delta/2,delta/2)
```

```
dim.S360
```

```
## [1] 140 120 1
```

```
range(ceiling((runif(1e5, xlim[1], xlim[2])-xlim[1])/delta))
```

```
## [1] 1 120
```

```
range(ceiling((ylim[2]-runif(1e5, ylim[1], ylim[2]))/delta))
```

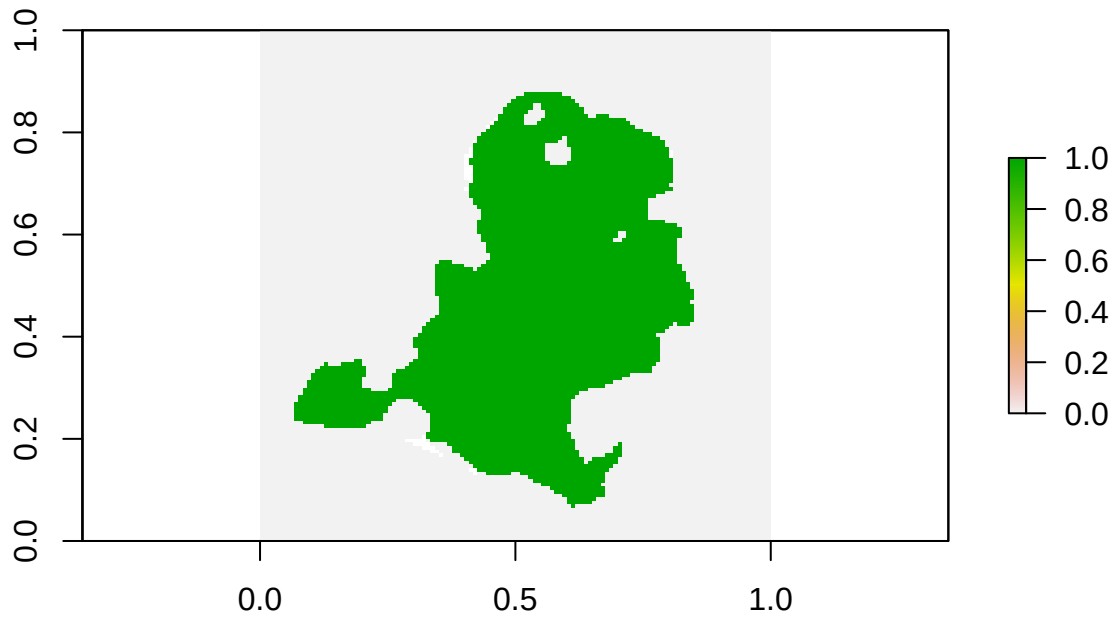
```
## [1] 1 140
```

```
S <- matrix(S360dat$habitat, dim.S360[1], dim.S360[2],
            dimnames=list(yseq, xseq), byrow=TRUE)
str(S)
```

```
## int [1:140, 1:120] 0 0 0 0 0 0 0 0 0 0 ...
```

```
## - attr(*, "dimnames")=List of 2
## ..$ : chr [1:140] "3624155" "3623795" "3623435" "3623075" ...
## ..$ : chr [1:120] "241404" "241764" "242124" "242484" ...
```

```
plot(raster(S))
```

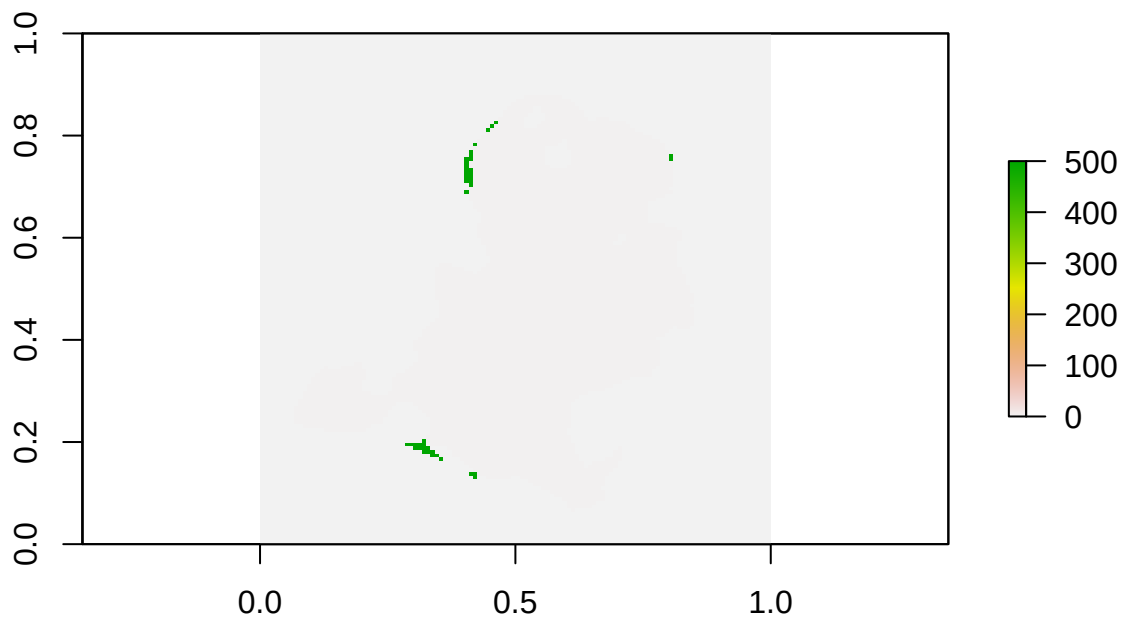


```
#Area of the state space
sum(S, na.rm=TRUE)*delta^2 / 1e6
```

```
## [1] 648
```

```
Sarea <- sum(S, na.rm=TRUE)*delta^2 / 1e6
```

```
# Where are these NA values??
Stmp <- S
Stmp[is.na(Stmp)] <- 500
plot(raster(Stmp))
```



```
# Better to assume they're NOT habitat than to assume that they are
# Recode them to 0's
S[is.na(S)] <- 0
```

The NA entries that caused problems for **JAGS** seem to be only along the periphery of the state space. Seems likely that these crop up because of slightly different positions of the traps which meant that the 4km buffer would've picked up a few additional cells in the raster.

Fit the model

The analysis in **JAGS** is below. First we prepare the data and create a function to set initial parameter values.

Prepare Data

```
## JAGS
library(rjags)

### Put the data in a list
jd <- list(yF=yF, yF.zero=matrix(0, Mf, nYears),
           yM=yM, yM.zero=matrix(0, Mm, nYears),
```

```

zF=zFdata.aug,
zM=zMdata.aug,
nFem=nFem, nMale=nMale,
Mf=Mf, Mm=Mm,
nTrapsYr=as.integer(nTrapsYr),
nWeeks=nWeeks, nYears=nYears,
oper.traps=oper.traps,
prevcapF=prevcap.f+1,
prevcapM=prevcap.m+1,
S=S, delta=delta,
onesF=rep(1, Mf),
onesM=rep(1, Mm),
x=traps, xlim=xlim, ylim=ylim,
poly.area=Sarea)

```

```
str(jd)
```

```

## List of 24
## $ yF      : num [1:88, 1:154, 1:7, 1:2] 0 0 0 0 0 0 0 0 0 0 0 ...
## $ yF.zero  : num [1:500, 1:2] 0 0 0 0 0 0 0 0 0 0 0 ...
## $ yM      : num [1:77, 1:154, 1:7, 1:2] 0 0 0 0 0 0 0 0 0 0 0 ...
## $ yM.zero  : num [1:500, 1:2] 0 0 0 0 0 0 0 0 0 0 0 ...
## $ zF      : num [1:500, 1:2] 1 1 1 1 NA 1 1 1 1 1 ...
## $ zM      : num [1:500, 1:2] NA 1 1 1 1 1 1 1 1 1 ...
## $ nFem    : int 88
## $ nMale   : int 77
## $ Mf      : num 500
## $ Mm      : num 500
## $ nTrapsYr : int [1:2] 130 138
## $ nWeeks   : int 7
## $ nYears   : int 2
## $ oper.traps: int [1:154, 1:2] 1 2 3 4 5 6 7 8 9 10 ...
## $ prevcapF : num [1:88, 1:154, 1:7, 1:2] 1 1 1 1 1 1 1 1 1 1 ...
## $ prevcapM : num [1:77, 1:154, 1:7, 1:2] 1 1 1 1 1 1 1 1 1 1 ...
## $ S        : num [1:140, 1:120] 0 0 0 0 0 0 0 0 0 0 ...
## ..- attr(*, "dimnames")=List of 2
## .. ..$ : chr [1:140] "3624155" "3623795" "3623435" "3623075" ...
## .. ..$ : chr [1:120] "241404" "241764" "242124" "242484" ...
## $ delta    : num 360
## $ onesF    : num [1:500] 1 1 1 1 1 1 1 1 1 1 ...
## $ onesM    : num [1:500] 1 1 1 1 1 1 1 1 1 1 ...
## $ x        : 'data.frame': 154 obs. of 2 variables:
## ..$ East : num [1:154] 248187 255814 248920 250904 252875 ...
## ..$ North: num [1:154] 3586614 3587061 3588050 3587890 3588670 ...
## $ xlim     : num [1:2] 241224 284424
## $ ylim     : num [1:2] 3573935 3624335
## $ poly.area : num 648

```

```
#### Function to generate initial values
```

```

ji <- function() {
  nFem <- jd$nFem
  nMale <- jd$nMale
  Mf <- jd$Mf

```

```

Mm <- jd$Mm
nYears <- jd$nYears
nWeeks <- jd$nWeeks
traps <- jd$x
nTraps <- nrow(traps)

#AC and z for Females
zFi <- matrix(0, Mf, nYears)
sFi1 <- traps[sample.int(nTraps, size=Mf, replace=TRUE,
                        prob=traps.in),]
sFi <- as.matrix(sFi1)
for(i in 1:nFem) {
  zFi[i,first.detF[i]:last.detF[i]] <- NA
  traplocs.i <- (rowSums(jd$yF[i,,])>0) * traps.in
  if(all(!traplocs.i))
    next
  sFi[i,] <- colMeans(traps[which(traplocs.i == 1),,drop=FALSE])
  #Check that the average detection location is in habitat
  testAC <- 1 == jd$S[round((jd$ylim[2]-sFi[i,2])/jd$delta+0.5),
                    round((sFi[i,1]-jd$xlim[1])/jd$delta+0.5)]
  S360hab <- S360dat[which(S360dat$habitat == 1),c("x","y")]
  if(!testAC) {
    tmp.dist <- sqrt((S360hab$x-sFi[i,1])^2 +
                    (S360hab$y-sFi[i,2])^2)
    sFi[i,1] <- S360hab$x[which(tmp.dist == min(tmp.dist))]
    sFi[i,2] <- S360hab$y[which(tmp.dist == min(tmp.dist))]
    rm(tmp.dist)
  }
  rm(testAC)
}

#AC and z for males
zM1 <- matrix(0, Mm, nYears)
sM11 <- traps[sample.int(nTraps, size=Mm, replace=TRUE,
                        prob=traps.in),]
sM1 <- as.matrix(sM11)
for(i in 1:nMale) {
  zM1[i,first.detM[i]:last.detM[i]] <- NA
  traplocs.i <- (rowSums(jd$yM[i,,])>0) * traps.in
  if(all(!traplocs.i))
    next
  sM1[i,] <- colMeans(traps[which(traplocs.i == 1),,drop=FALSE])
  #Check that the average detection location is in habitat
  testAC <- 1 == jd$S[round((jd$ylim[2]-sM1[i,2])/jd$delta+0.5),
                    round((sM1[i,1]-jd$xlim[1])/jd$delta+0.5)]
  S360hab <- S360dat[which(S360dat$habitat == 1),c("x","y")]
  if(!testAC) {
    tmp.dist <- sqrt((S360hab$x-sM1[i,1])^2 +
                    (S360hab$y-sM1[i,2])^2)
    sM1[i,1] <- S360hab$x[which(tmp.dist == min(tmp.dist))]
    sM1[i,2] <- S360hab$y[which(tmp.dist == min(tmp.dist))]
    rm(tmp.dist)
  }
}

```

```

    rm(testAC)
  }

  p0i <- array(NA, c(2, nYears, 2))
  p0i[,2] <- matrix(runif(2*nYears), 2, nYears)
  out <- list(zF=zFi, zM=zMi, sF=sFi, sM=sMi, p0st=p0i,
             sigma=runif(2, 3000, 4000),
             kappa=runif(2, 0.5, 1),
             psi=matrix(c(runif(2), rep(NA, nYears)), 2, nYears),
             phi=runif(2), ### Added this, we didn't draw init. phi before?
             gamma=runif(1, 0.2, 0.3))
  out$.RNG.name <- "base::Mersenne-Twister"
  out$.RNG.seed <- sample.int(1e6, 1)
  return(out)
}

str(ji())

```

```

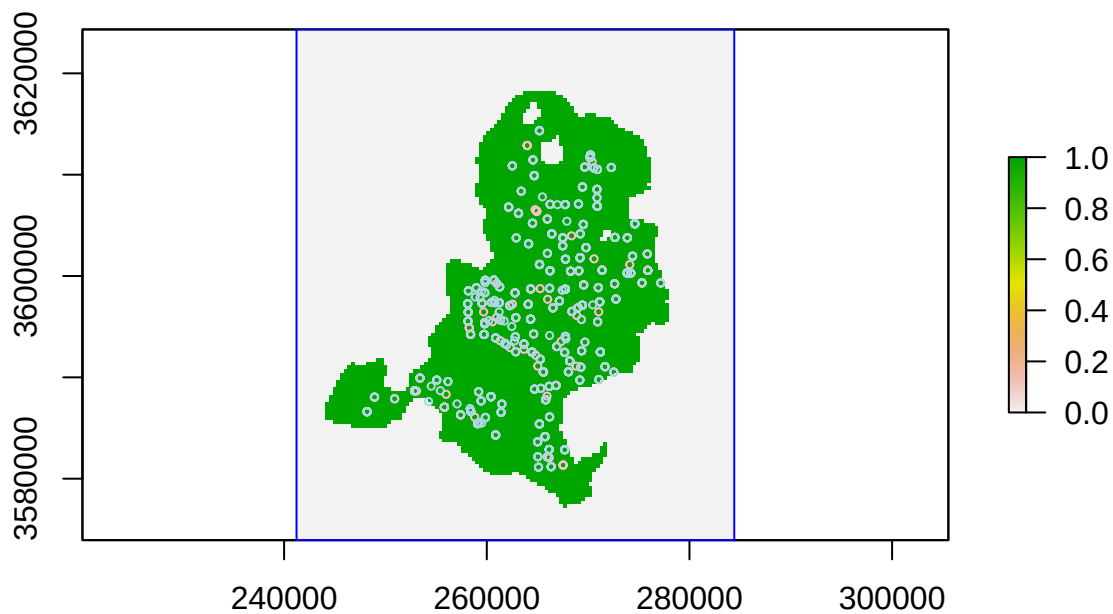
## List of 12
## $ zF      : num [1:500, 1:2] NA NA NA NA 0 NA NA NA NA NA ...
## $ zM      : num [1:500, 1:2] 0 NA NA NA NA NA NA NA NA NA ...
## $ sF      : num [1:500, 1:2] 260975 261320 265027 268814 262530 ...
## ..- attr(*, "dimnames")=List of 2
## .. ..$ : chr [1:500] "262817x3594007" "263988x3612881" "269565x3599125" "269051x3607111" ...
## .. ..$ : chr [1:2] "East" "North"
## $ sM      : num [1:500, 1:2] 265276 259335 261218 260762 262737 ...
## ..- attr(*, "dimnames")=List of 2
## .. ..$ : chr [1:500] "263132x3606186" "262817x3594007" "262817x3594007.1" "269051x3607111" ...
## .. ..$ : chr [1:2] "East" "North"
## $ p0st    : num [1:2, 1:2, 1:2] NA NA NA NA 0.091 ...
## $ sigma   : num [1:2] 3110 3595
## $ kappa   : num [1:2] 0.644 0.607
## $ psi     : num [1:2, 1:2] 0.786 0.971 NA NA
## $ phi     : num [1:2] 0.543 0.609
## $ gamma   : num 0.267
## $ .RNG.name: chr "base::Mersenne-Twister"
## $ .RNG.seed: int 4257

```

```

## Visualize initial values of activity centers
plot(raster(S, xmn=xlim[1], ymn=ylim[1], xmx=xlim[2], ymx=ylim[2]))
points(ji()$sF, pch = 1, cex = 0.5, col = "pink")
points(ji()$sM, pch= 1, cex = 0.5, col = "lightblue")
rect(xlim[1], ylim[1], xlim[2], ylim[2], border="blue")

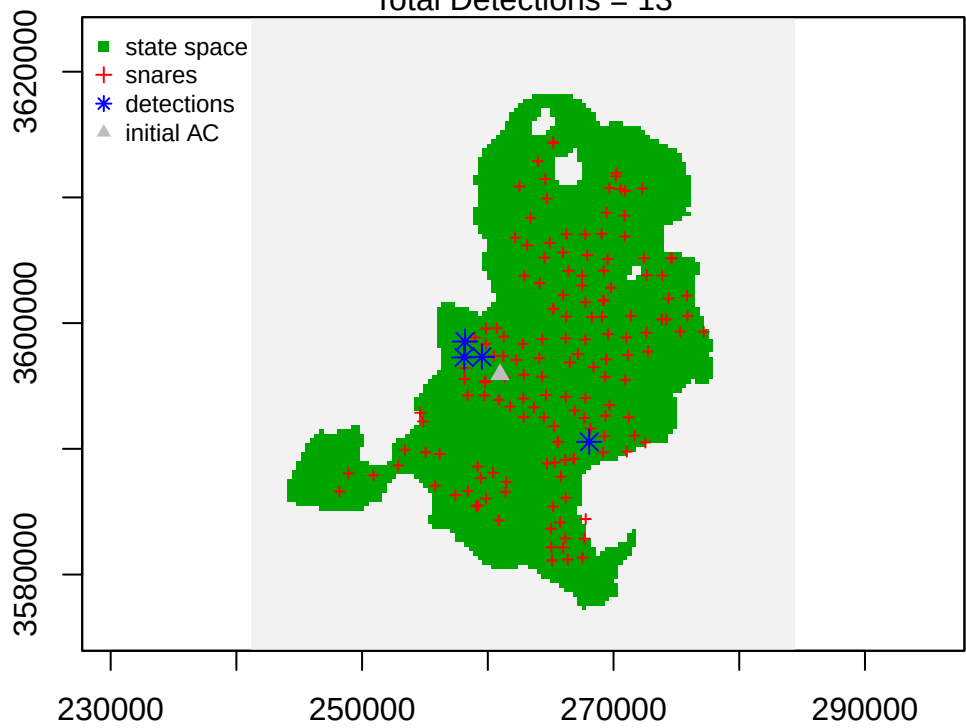
```



```
##Focus in on a few individuals with multiple detections
# is ji() placing initial activity centers in the right places?
par(mar = c(3,3,3,3))
closer.look <- which(rowSums(jd$yF) > 5)
for(i in closer.look) {
  tmp.s <- ji()$sF[i,]
  plot(raster(S, xmn=xlim[1], ymn=ylim[1], xmx=xlim[2], ymx=ylim[2]),
       main = paste0 ("Female ",i, " - ", "Clone ", Fch$clone[i]),
       legend = FALSE)
  mtext(text = paste("Total Detections =", sum(jd$yF[i,,])))
  points(traps, pch = 3, col = "red", cex = 0.5)
  points(jd$x[which(rowSums(jd$yF[i,,]) > 0)], pch = 8, col = "blue", cex = 1.2)
  points(x=tmp.s[1], y=tmp.s[2], pch = 17, col = "grey")
  legend("topleft", bty = "n", pch = c(15, 3, 8, 17), cex = 0.75,
        col = c(terrain.colors(10)[1], "red", "blue", "grey"),
        legend=c("state space", "snares", "detections", "initial AC"))
  rm(tmp.s)
}
```

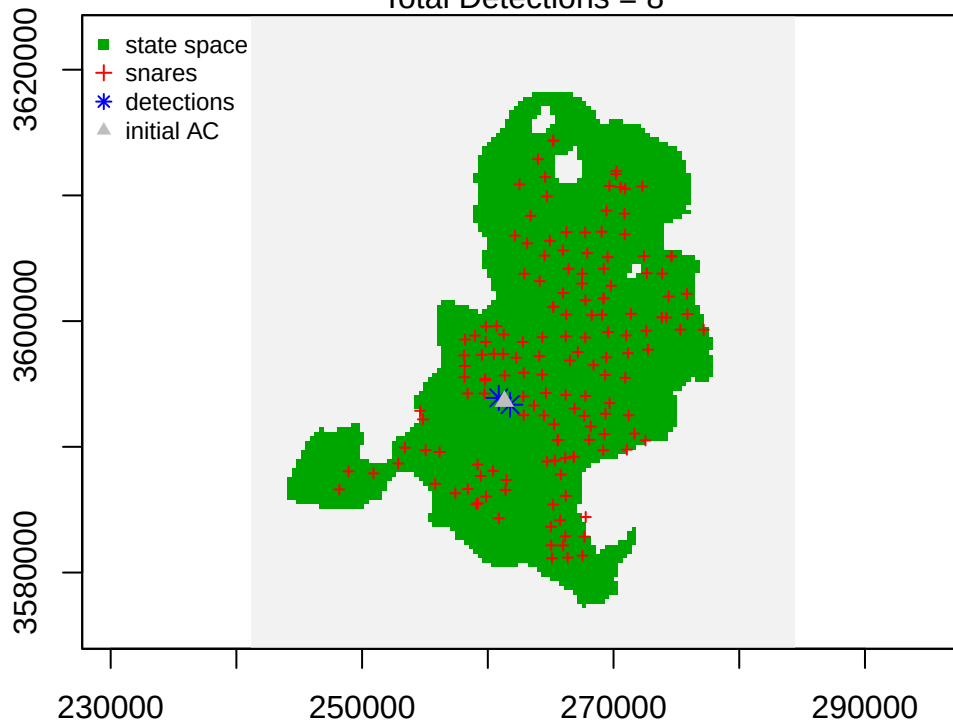
Female 1 – Clone 7

Total Detections = 13



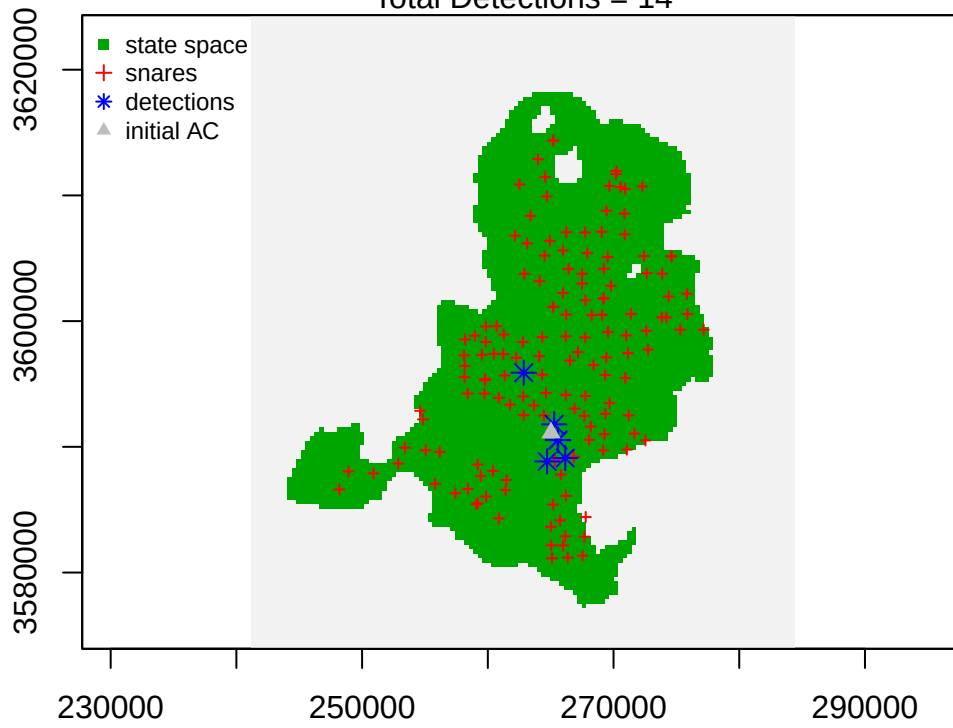
Female 2 – Clone 322

Total Detections = 8



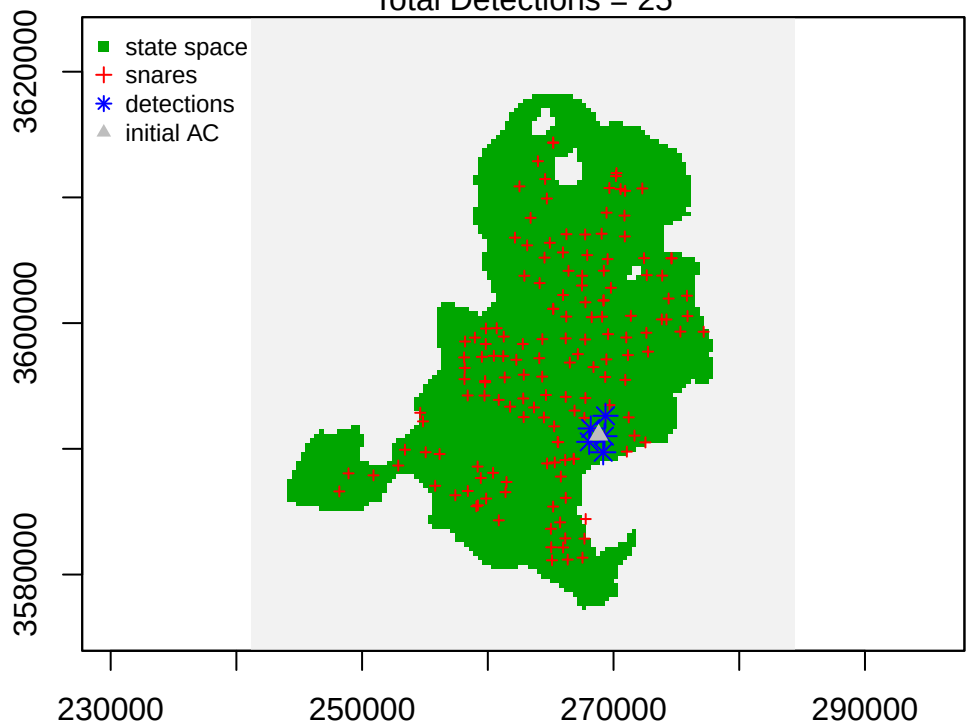
Female 3 – Clone 291

Total Detections = 14



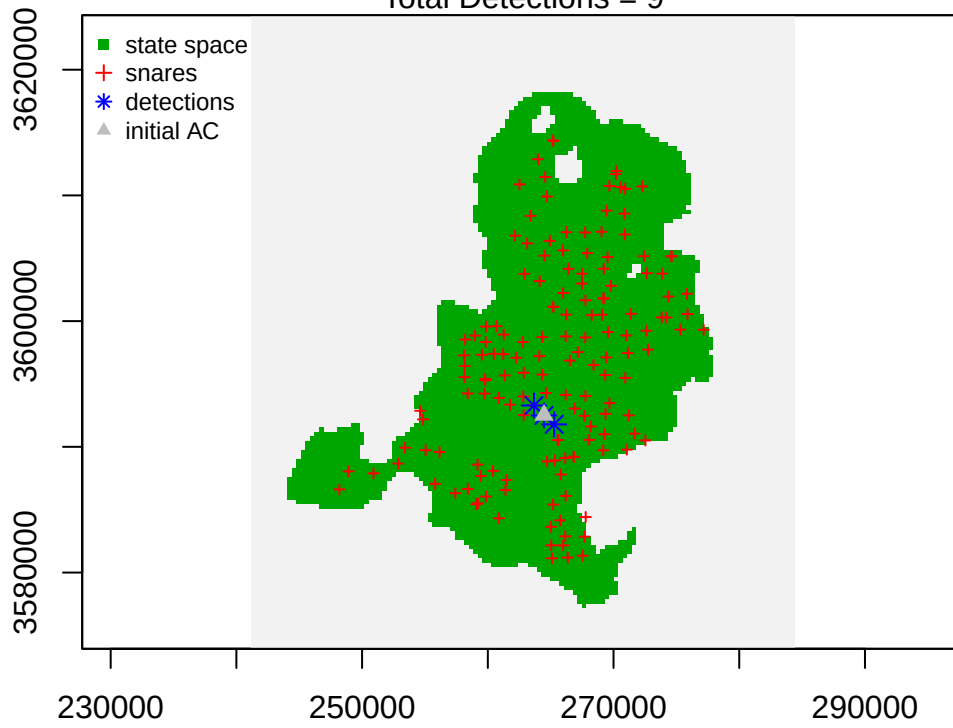
Female 4 – Clone 4

Total Detections = 25



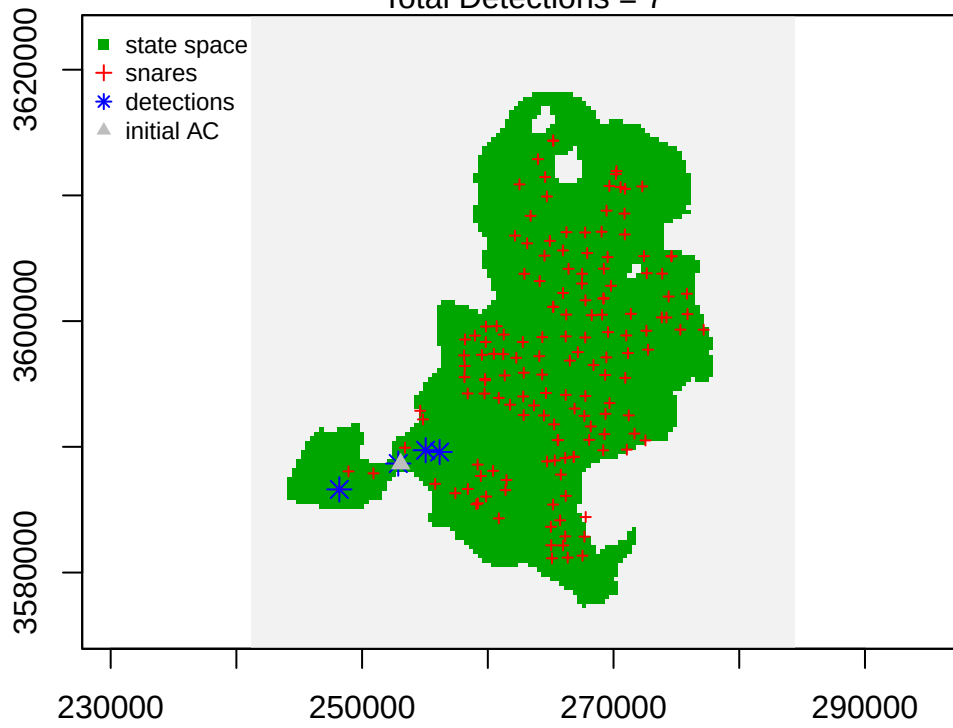
Female 6 – Clone 14

Total Detections = 9



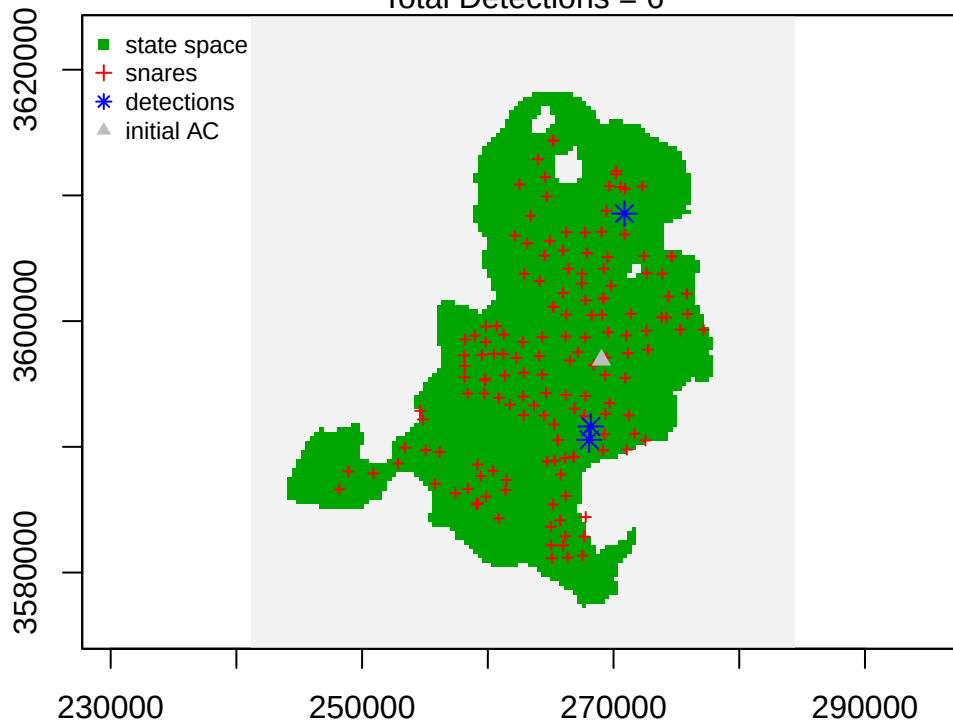
Female 8 – Clone 275

Total Detections = 7



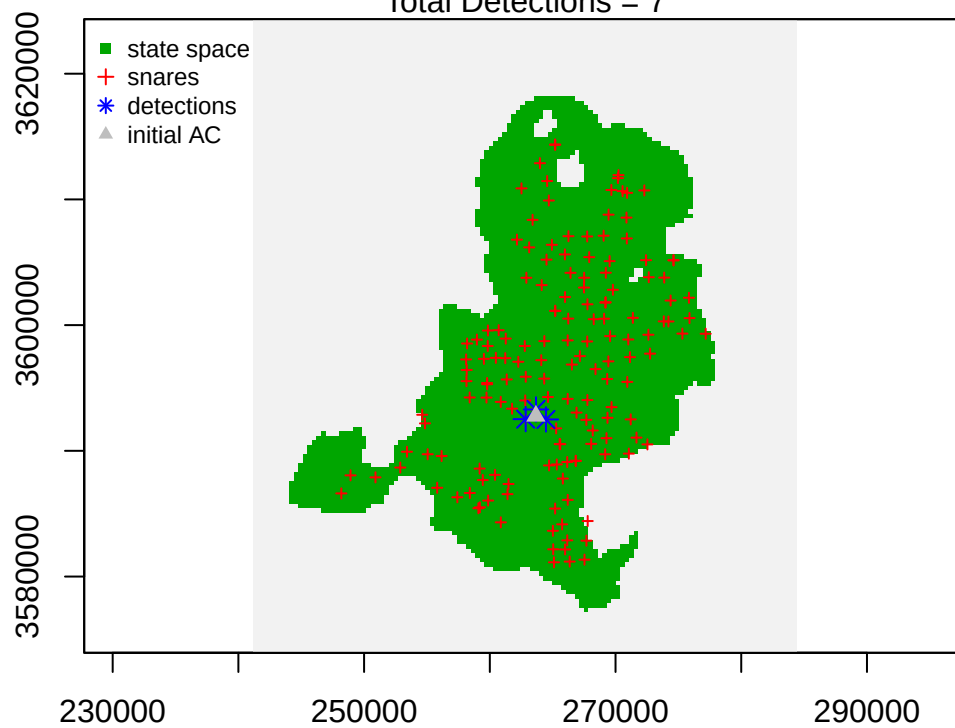
Female 9 – Clone 43

Total Detections = 6



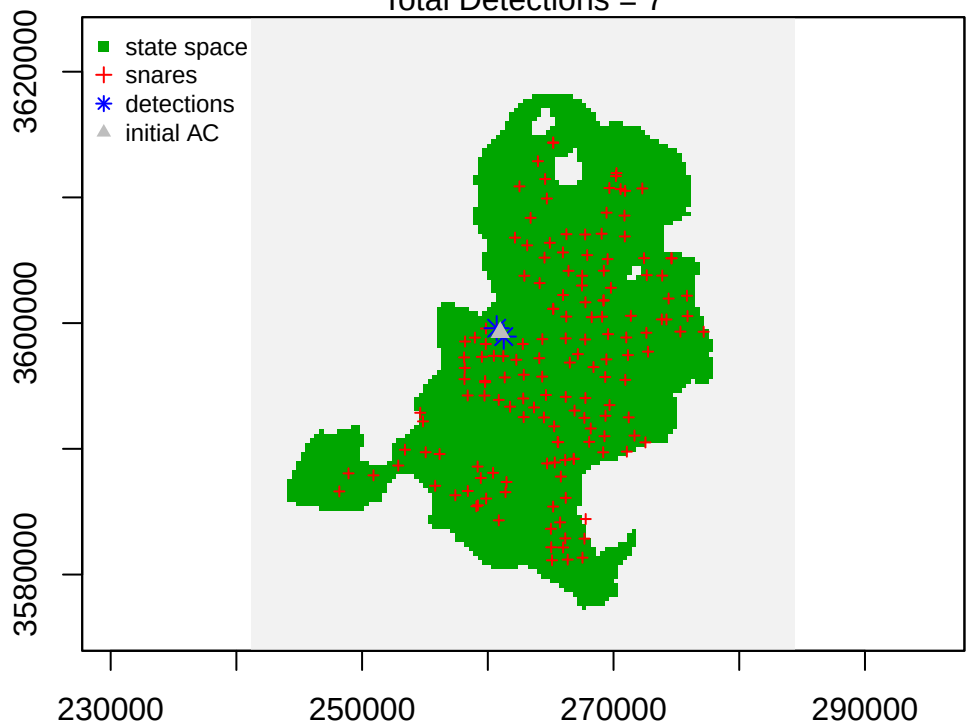
Female 10 – Clone 3

Total Detections = 7



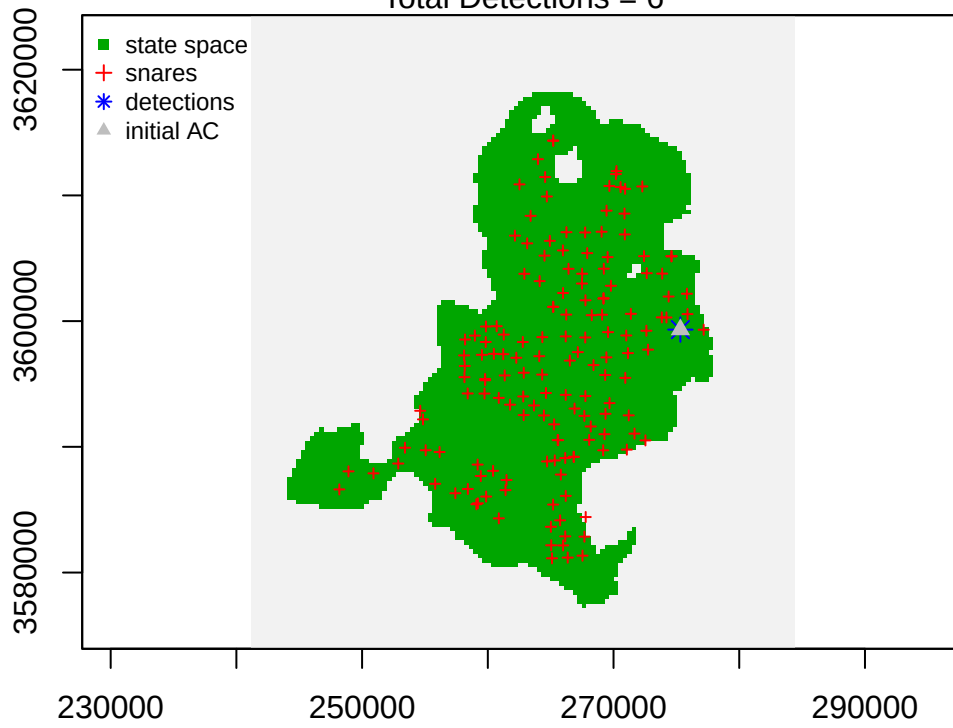
Female 11 – Clone 298

Total Detections = 7



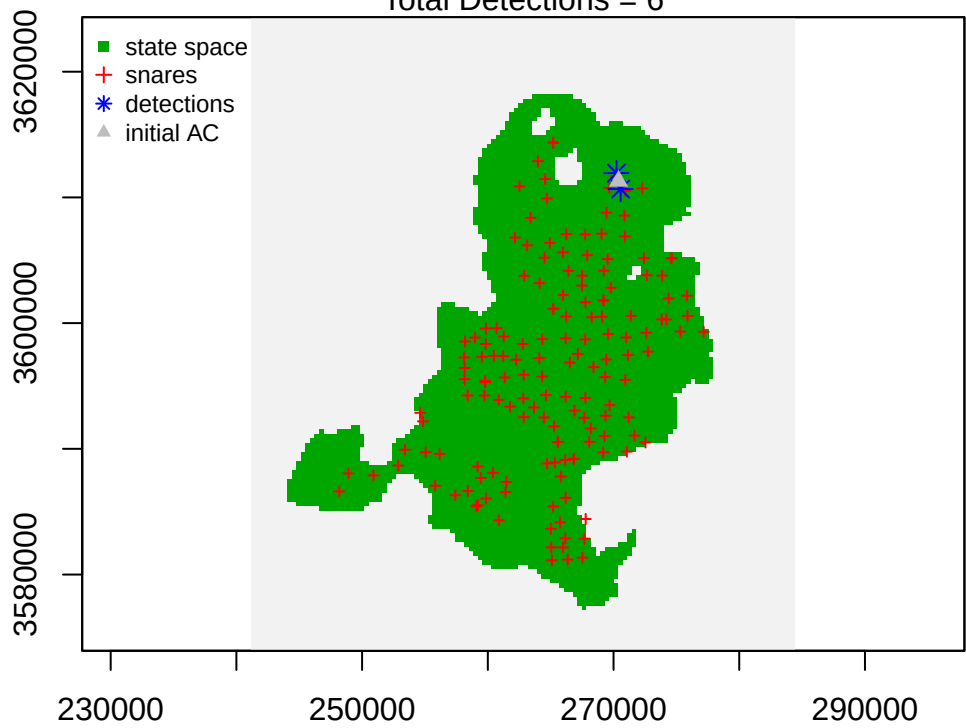
Female 14 – Clone 281

Total Detections = 6



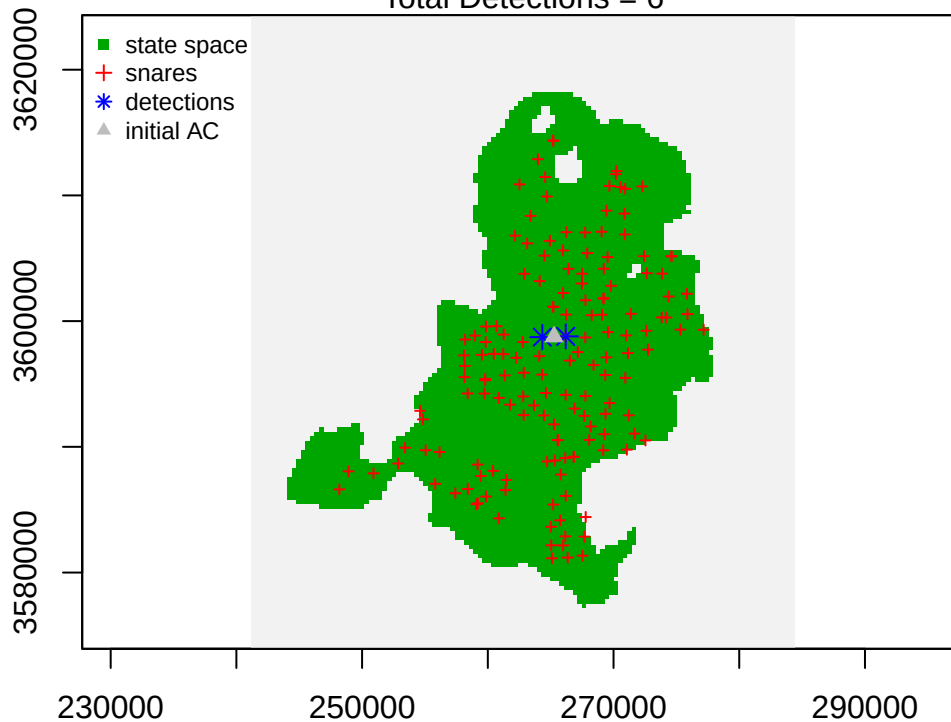
Female 20 – Clone 309

Total Detections = 6



Female 34 – Clone 277

Total Detections = 6



```
#### Parameters to monitor
```

```
jp <- c("p0st", "sigma", "kappa", "gamma", "phi", "psi",
        "EN", "N", "ENf", "Nf", "ENm", "Nm", "Ntotal", "totF", "totM",
        "ED", "RD", "EDf", "RDf", "EDm", "RDm",
        "survivingF", "survivingM", "recruitF", "recruitM",
        "lambdaF", "lambdaM", "lambda")
```

Now to run the MCMC sampler. Note that several steps from the full [fit.R](#) script from [Hooker et al. \(2020\)](#) were skipped below. These steps involved calculation of WAIC or recording information needed for forecasting future population sizes under the model. Here, we simplify to two steps:

- i. Adaptive phase sampling - 1000 iterations per chain
- ii. Collect samples from the posterior - 10,000 iterations per chain

```
## Models in parallel
```

```
library(parallel)
```

```
(nCores <- max(1, min(detectCores()-1, 4)))
```

```
## [1] 4
```

```
c11 <- makeCluster(nCores)
```

```
clusterExport(c11, c("jd", "ji", "jp",
```

```

        "traps.in", "S360dat",
        "first.detF", "last.detF", "first.detM", "last.detM"))

## Compile and adapt
system.time({
  adapt <- clusterEvalQ(cl1, {
    library(rjags)
    load.module("dic")
    load.module("glm")
    mod <- jags.model(file="../input/CGA_SCR.jag",
                      data=jd, inits=ji,
                      n.chains=1, n.adapt=1000)

    return(mod)
  })
})

```

```

##      user      system elapsed
##    0.070      0.079 6280.534

```

```

save(adapt, file="adapt-bothSexes.gzip")

```

```

## Draw posterior samples, including the latent variables needed
## for forecasting
system.time({
  samples <- clusterEvalQ(cl1, {
    mod1 <- coda.samples(model=mod,
                        variable.names=c(jp, "deviance",
                                          "zF", "aF", "sF",
                                          "zM", "aM", "sM"),
                        n.iter=10000, thin=1)

    return(as.mcmc(mod1))
  })
})

```

```

##      user      system elapsed
##    3.440      2.558 51460.073

```

```

save(samples, file="samples-bothSexes.gzip")

```

```

stopCluster(cl1)

```

Posterior Samples

Now to plot a couple of posteriors and save the workspace for further analysis.

```

mc1 <- as.mcmc.list(samples)

focal.parms <- c("Nf[1]", "Nf[2]", "Nm[1]", "Nm[2]",
                "sigma[1]", "sigma[2]", "phi[1]", "phi[2]", "gamma",
                "lambdaF", "lambdaM", "lambda",

```

```

      "p0st[1,1,1]", "p0st[1,1,2]", "p0st[1,2,1]", "p0st[1,2,2]",
      "p0st[2,1,1]", "p0st[2,1,2]", "p0st[2,2,1]", "p0st[2,2,2]",
      "kappa[1]", "kappa[2]")

# Posterior summary
summary(mc1[,focal.parms])

##
## Iterations = 1001:11000
## Thinning interval = 1
## Number of chains = 4
## Sample size per chain = 10000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##              Mean          SD Naive SE Time-series SE
## Nf[1]          1.144e+02 1.983e+01 9.917e-02      5.847e-01
## Nf[2]          1.460e+02 1.603e+01 8.013e-02      3.524e-01
## Nm[1]          9.331e+01 1.008e+01 5.038e-02      1.210e-01
## Nm[2]          5.804e+01 6.818e+00 3.409e-02      1.025e-01
## sigma[1]       2.226e+03 1.387e+02 6.935e-01      2.702e+00
## sigma[2]       2.437e+03 1.168e+02 5.840e-01      1.706e+00
## phi[1]         8.371e-01 7.734e-02 3.867e-04      1.418e-03
## phi[2]         2.868e-01 6.305e-02 3.152e-04      6.569e-04
## gamma          3.768e-01 1.303e-01 6.514e-04      3.577e-03
## lambdaF        1.298e+00 1.651e-01 8.256e-04      3.848e-03
## lambdaM        6.280e-01 9.154e-02 4.577e-04      1.304e-03
## lambda         9.897e-01 9.991e-02 4.995e-04      1.910e-03
## p0st[1,1,1]    1.821e-02 3.790e-03 1.895e-05      8.536e-05
## p0st[1,1,2]    4.702e-01 7.056e-02 3.528e-04      1.018e-03
## p0st[1,2,1]    1.049e-02 2.015e-03 1.007e-05      4.035e-05
## p0st[1,2,2]    2.703e-01 2.883e-02 1.441e-04      3.438e-04
## p0st[2,1,1]    2.204e-02 3.310e-03 1.655e-05      3.998e-05
## p0st[2,1,2]    1.899e-01 3.114e-02 1.557e-04      4.777e-04
## p0st[2,2,1]    2.896e-02 4.632e-03 2.316e-05      6.538e-05
## p0st[2,2,2]    2.477e-01 3.013e-02 1.507e-04      3.836e-04
## kappa[1]       3.907e-02 7.702e-03 3.851e-05      1.674e-04
## kappa[2]       1.179e-01 1.936e-02 9.682e-05      3.341e-04
##
## 2. Quantiles for each variable:
##
##              2.5%       25%       50%       75%       97.5%
## Nf[1]          8.100e+01 1.000e+02 1.130e+02 1.270e+02 1.580e+02
## Nf[2]          1.180e+02 1.350e+02 1.450e+02 1.560e+02 1.810e+02
## Nm[1]          7.600e+01 8.600e+01 9.200e+01 9.900e+01 1.160e+02
## Nm[2]          4.700e+01 5.300e+01 5.700e+01 6.200e+01 7.300e+01
## sigma[1]       1.977e+03 2.129e+03 2.218e+03 2.314e+03 2.518e+03
## sigma[2]       2.221e+03 2.355e+03 2.431e+03 2.513e+03 2.678e+03
## phi[1]         6.681e-01 7.878e-01 8.442e-01 8.936e-01 9.670e-01
## phi[2]         1.746e-01 2.420e-01 2.827e-01 3.273e-01 4.220e-01
## gamma          1.671e-01 2.832e-01 3.610e-01 4.547e-01 6.704e-01
## lambdaF        1.022e+00 1.181e+00 1.281e+00 1.396e+00 1.671e+00

```

```
## lambdaM      4.673e-01 5.638e-01 6.214e-01 6.854e-01 8.247e-01
## lambda      8.112e-01 9.205e-01 9.834e-01 1.052e+00 1.203e+00
## p0st[1,1,1] 1.181e-02 1.550e-02 1.787e-02 2.054e-02 2.657e-02
## p0st[1,1,2] 3.445e-01 4.213e-01 4.656e-01 5.148e-01 6.215e-01
## p0st[1,2,1] 7.043e-03 9.065e-03 1.031e-02 1.174e-02 1.489e-02
## p0st[1,2,2] 2.174e-01 2.500e-01 2.691e-01 2.890e-01 3.299e-01
## p0st[2,1,1] 1.611e-02 1.971e-02 2.186e-02 2.416e-02 2.906e-02
## p0st[2,1,2] 1.349e-01 1.680e-01 1.881e-01 2.095e-01 2.567e-01
## p0st[2,2,1] 2.078e-02 2.570e-02 2.867e-02 3.191e-02 3.878e-02
## p0st[2,2,2] 1.924e-01 2.268e-01 2.461e-01 2.672e-01 3.106e-01
## kappa[1]    2.591e-02 3.364e-02 3.836e-02 4.386e-02 5.599e-02
## kappa[2]    8.453e-02 1.041e-01 1.164e-01 1.299e-01 1.598e-01
```

Check ESS

```
effectiveSize(mc1[,focal.parms])
```

```
##      Nf[1]      Nf[2]      Nm[1]      Nm[2]      sigma[1]      sigma[2]
## 1160.463 2093.608 6952.537 4467.120 2651.556 4694.637
## phi[1]   phi[2]   gamma   lambdaF   lambdaM   lambda
## 2973.439 9278.255 1331.186 1841.003 5085.733 2749.101
## p0st[1,1,1] p0st[1,1,2] p0st[1,2,1] p0st[1,2,2] p0st[2,1,1] p0st[2,1,2]
## 1974.487 4823.744 2498.340 7038.531 6909.455 4250.765
## p0st[2,2,1] p0st[2,2,2] kappa[1] kappa[2]
## 5030.239 6181.169 2115.809 3369.004
```

Check Rhat

```
gelman.diag(mc1[,focal.parms])
```

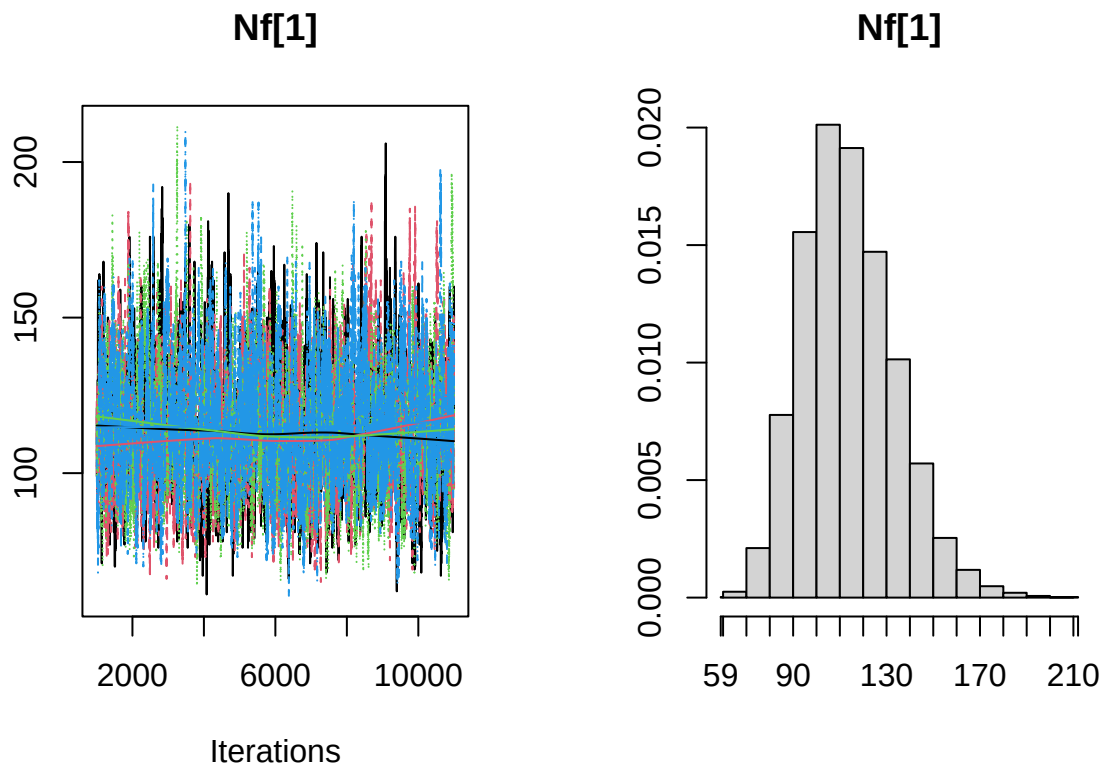
Potential scale reduction factors:

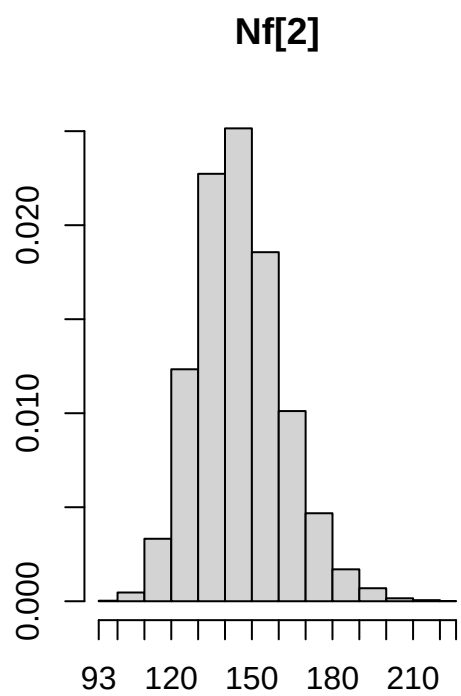
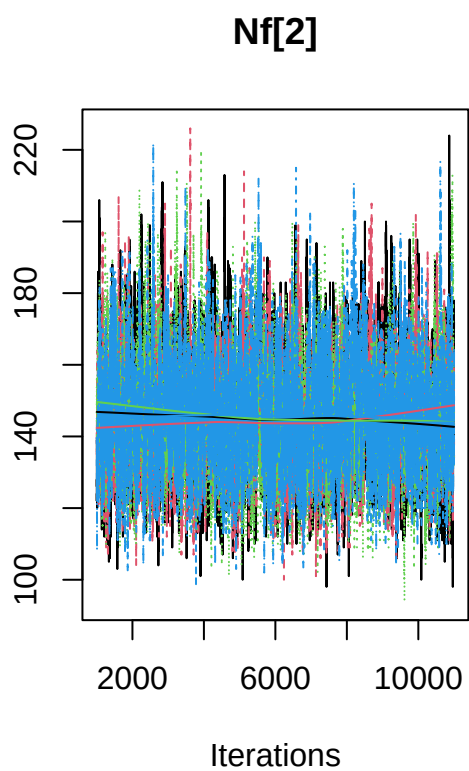
```
##
##      Point est. Upper C.I.
## Nf[1]      1.00      1.01
## Nf[2]      1.00      1.00
## Nm[1]      1.00      1.00
## Nm[2]      1.00      1.00
## sigma[1]   1.00      1.00
## sigma[2]   1.00      1.00
## phi[1]     1.00      1.00
## phi[2]     1.00      1.00
## gamma      1.00      1.01
## lambdaF    1.00      1.01
## lambdaM    1.00      1.00
## lambda     1.00      1.00
## p0st[1,1,1] 1.00      1.00
## p0st[1,1,2] 1.00      1.01
## p0st[1,2,1] 1.00      1.00
## p0st[1,2,2] 1.00      1.00
## p0st[2,1,1] 1.00      1.00
## p0st[2,1,2] 1.00      1.01
## p0st[2,2,1] 1.00      1.01
## p0st[2,2,2] 1.00      1.00
## kappa[1]   1.00      1.00
## kappa[2]   1.01      1.01
```

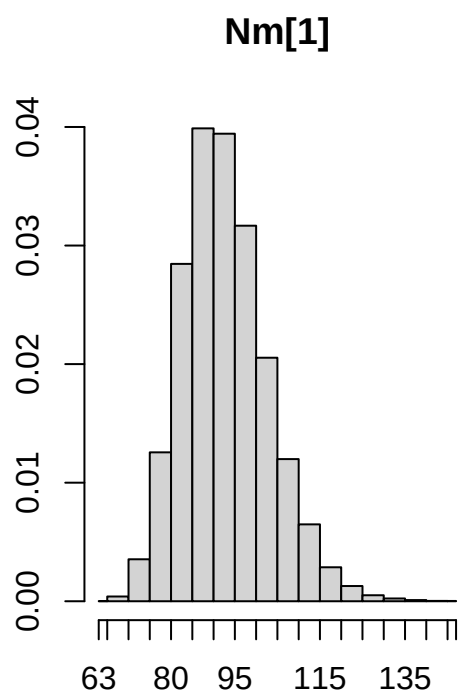
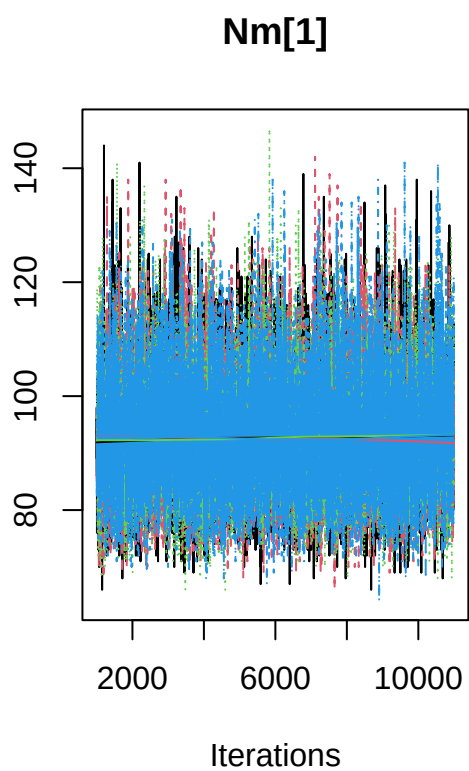
```
##
## Multivariate psrf
##
## 1.01
```

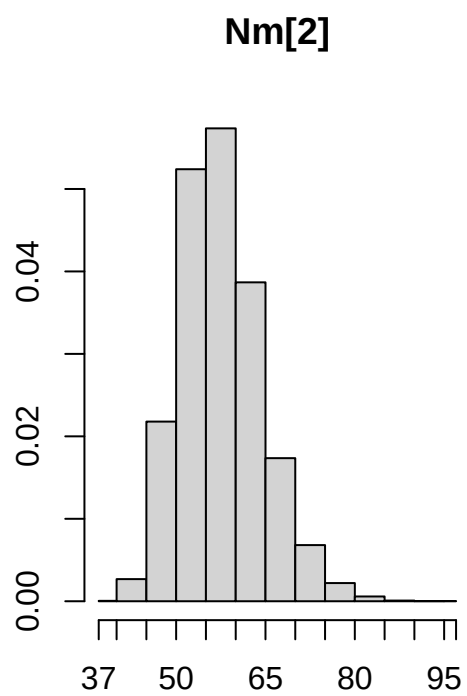
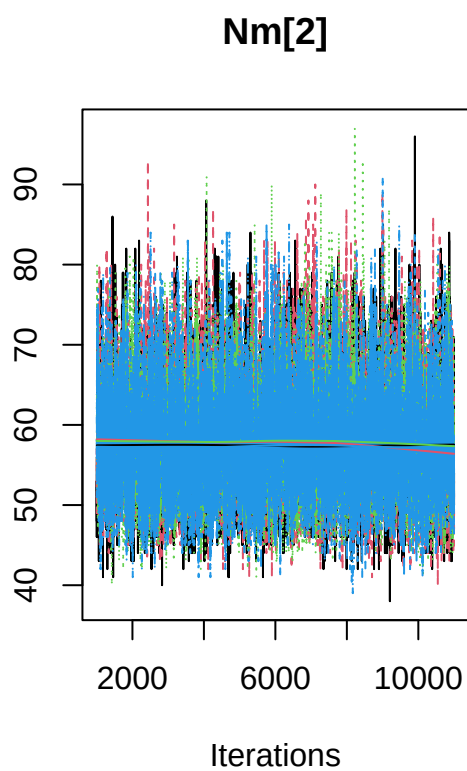
Now plot the posterior distributions for the model parameters

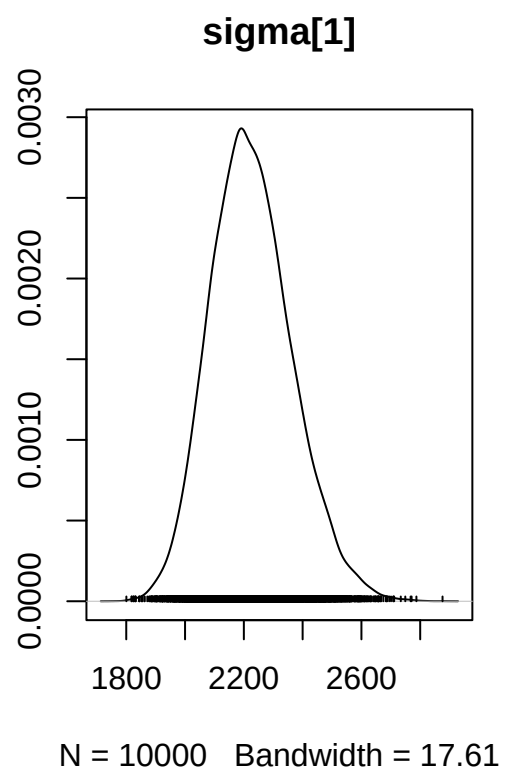
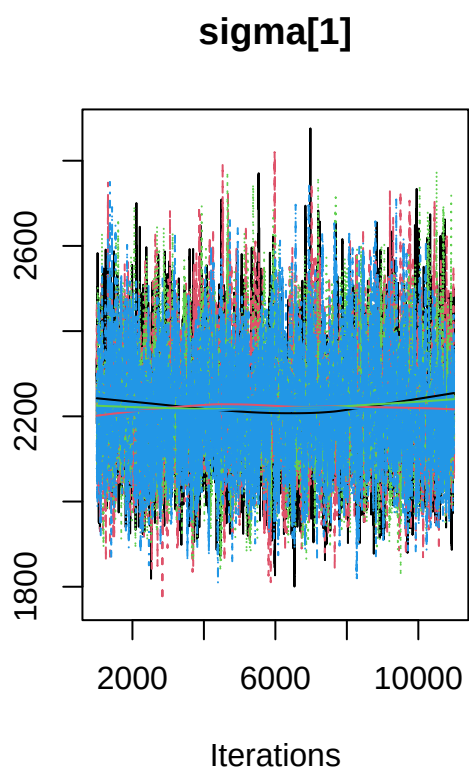
```
# Plot some posteriors
for(p in 1:length(focal.parms)) {
  plot(mc1[,focal.parms[p]], main = focal.parms[p])
}
```

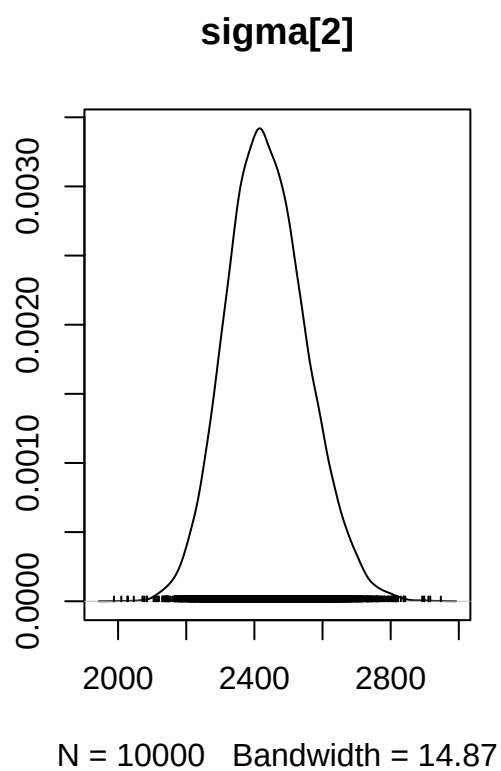
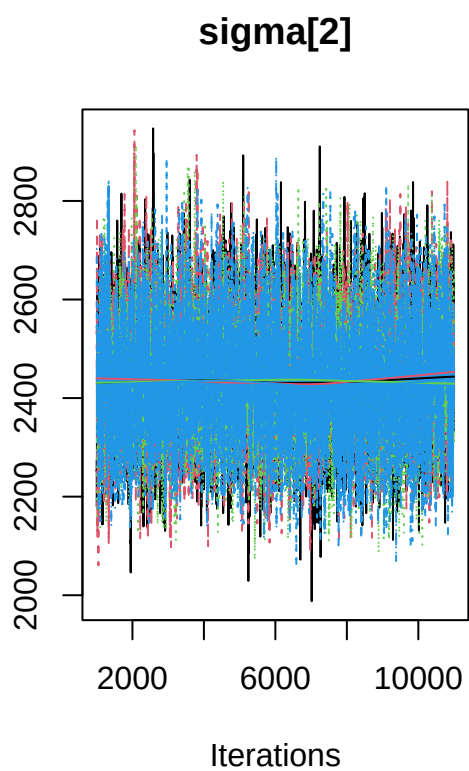


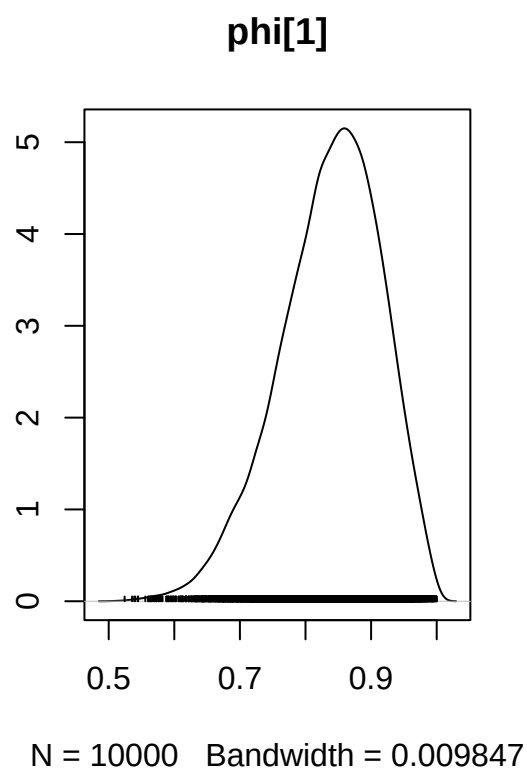
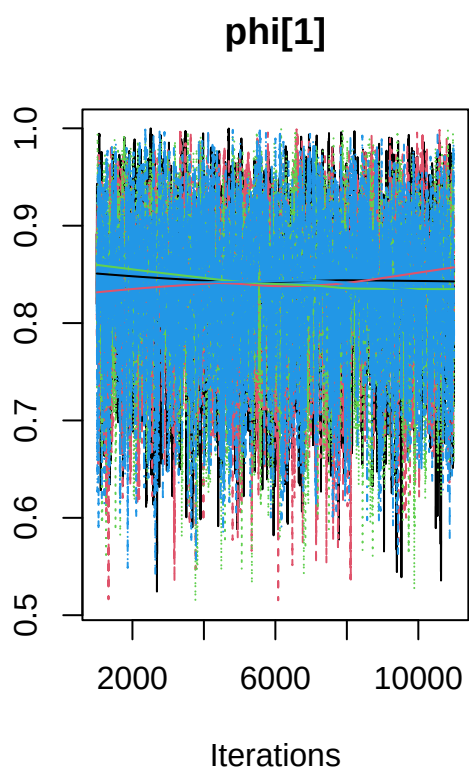


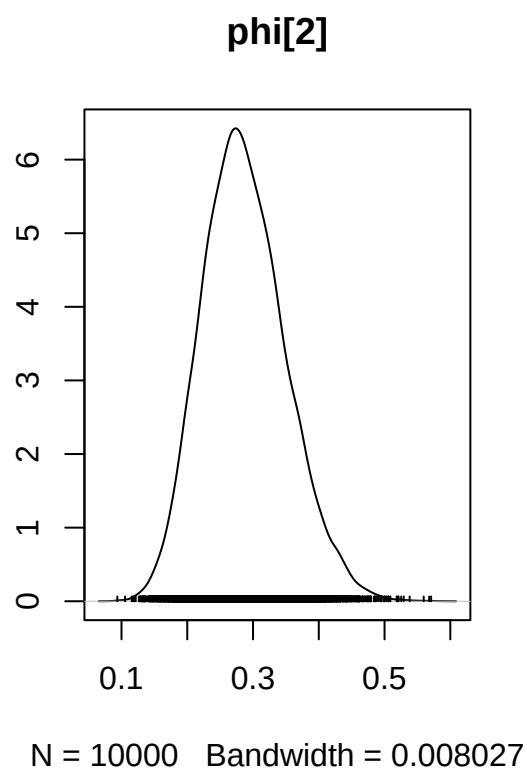
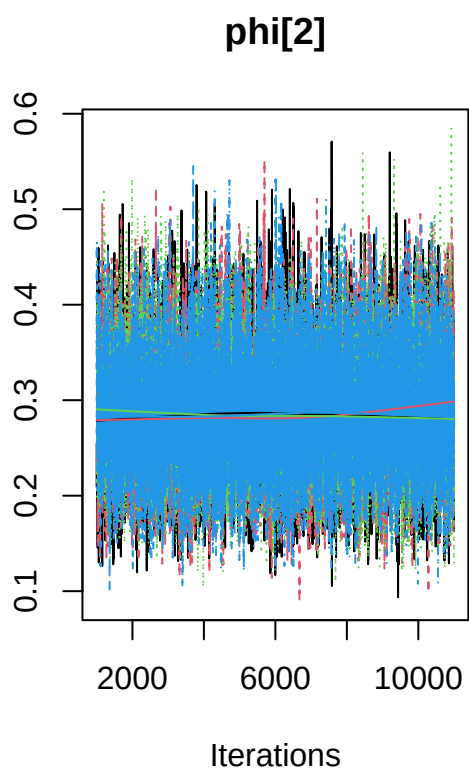


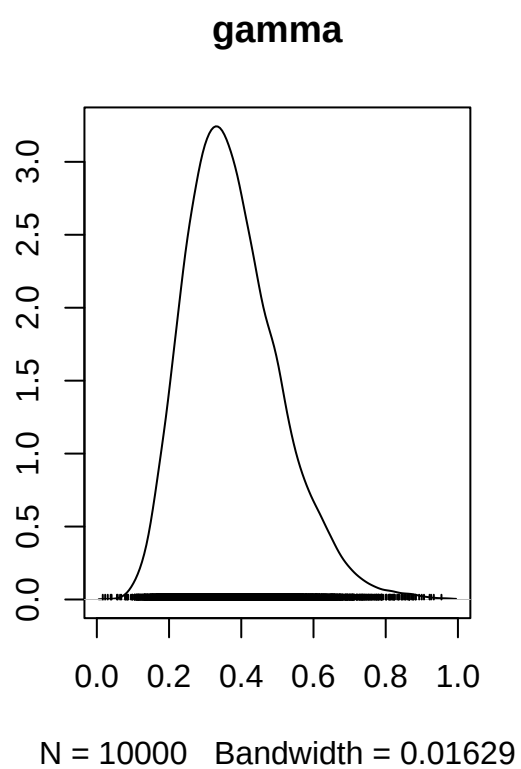
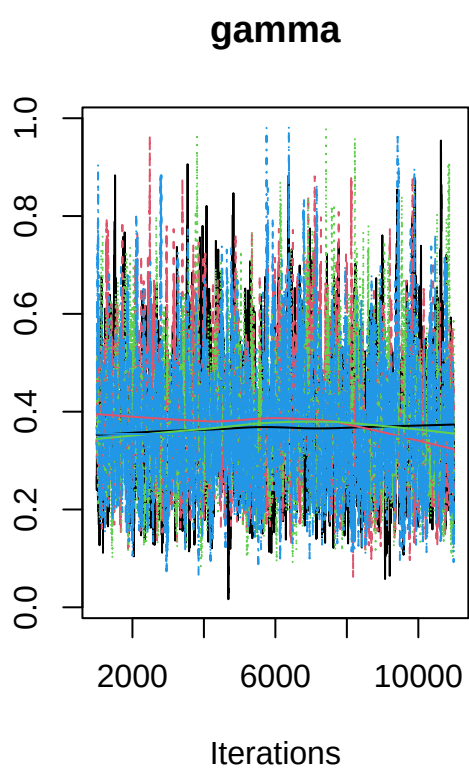




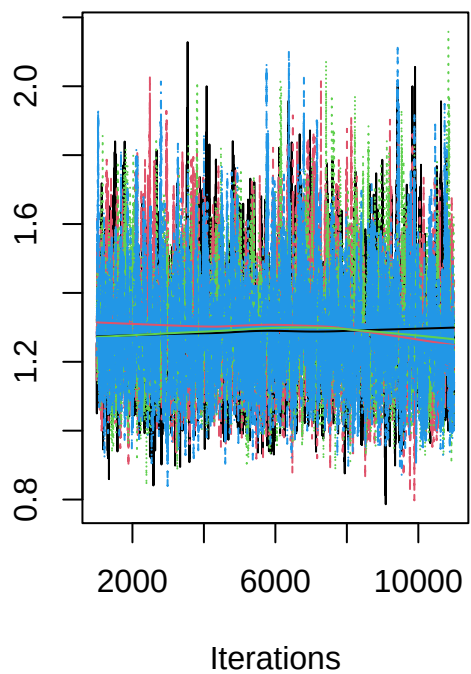




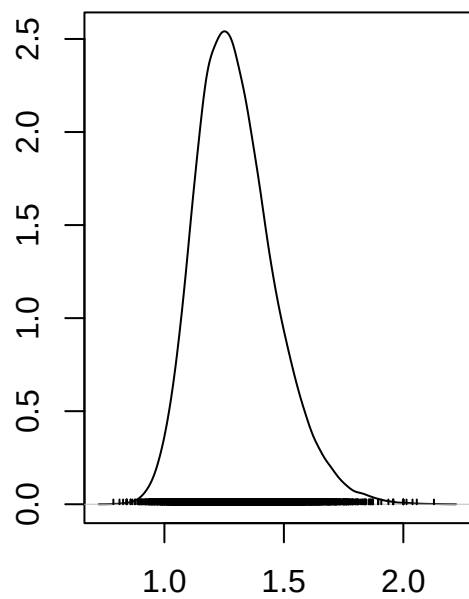




lambdaF

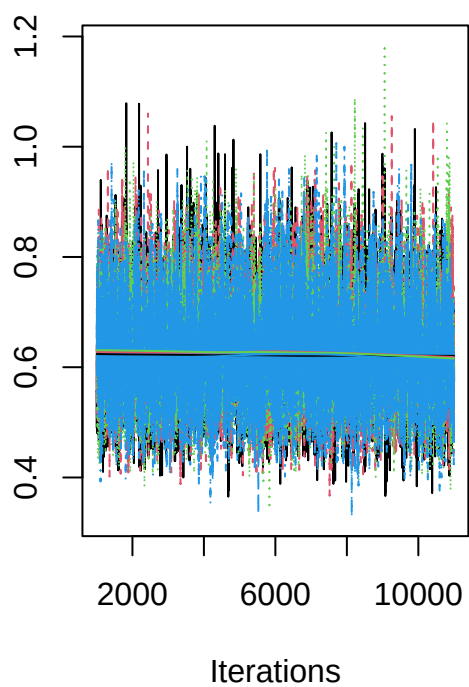


lambdaF

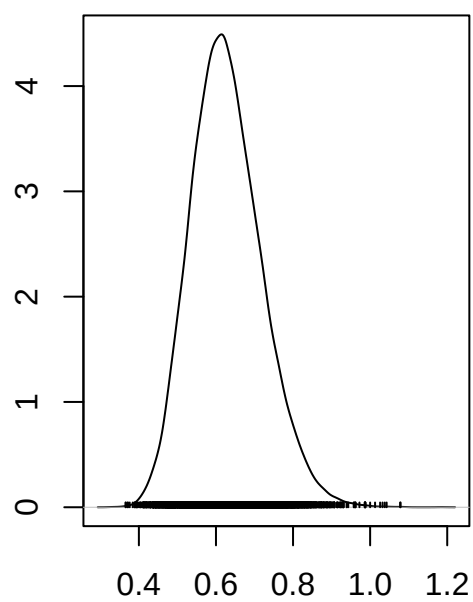


N = 10000 Bandwidth = 0.02043

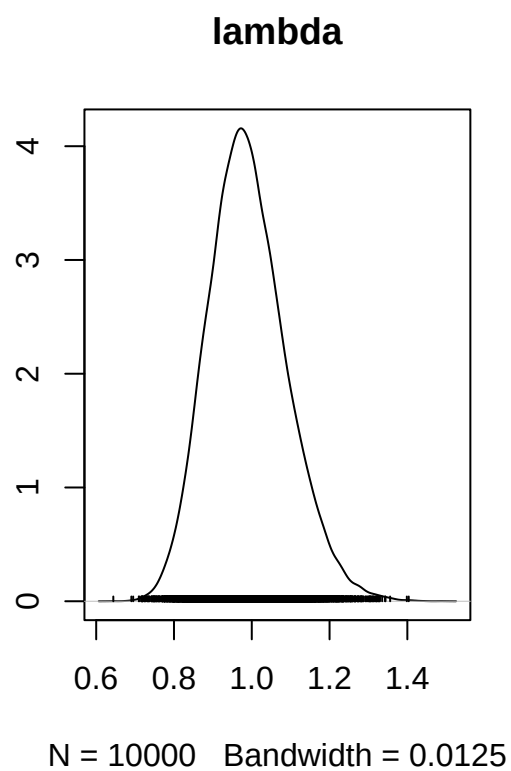
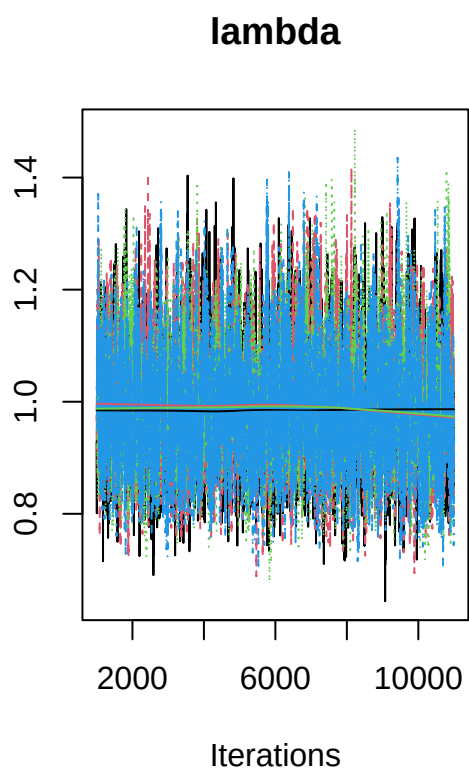
lambdaM

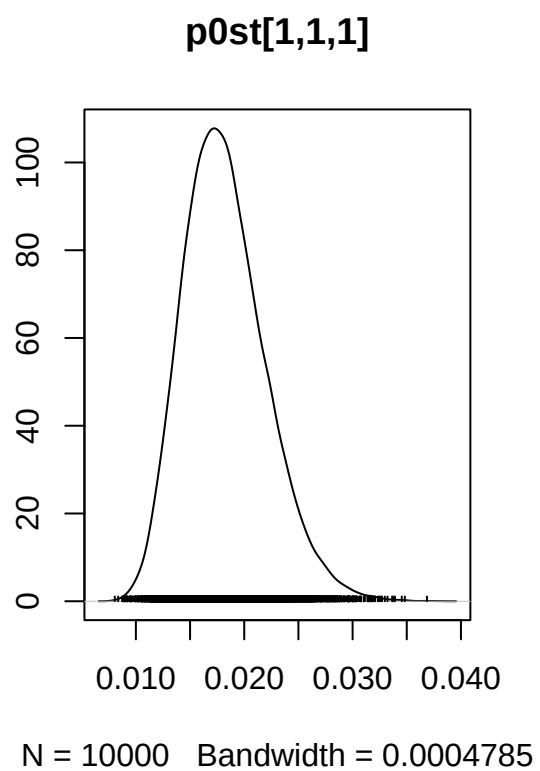
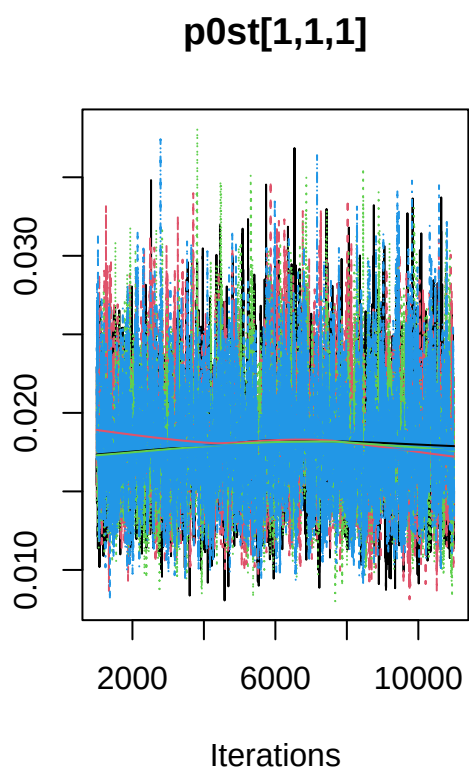


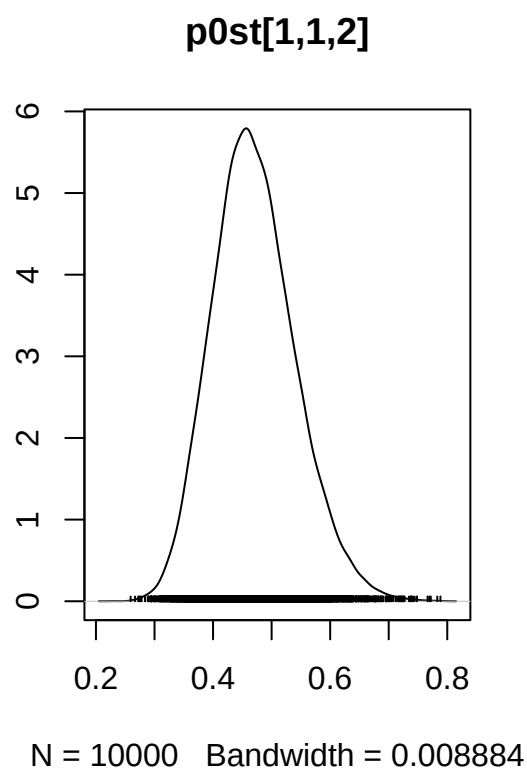
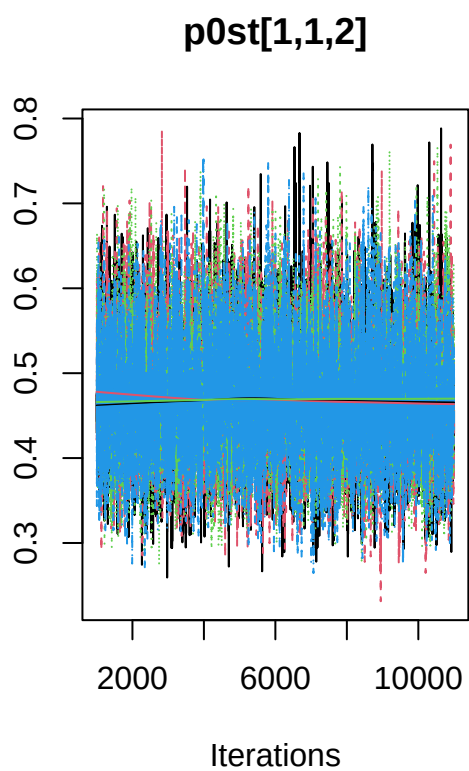
lambdaM

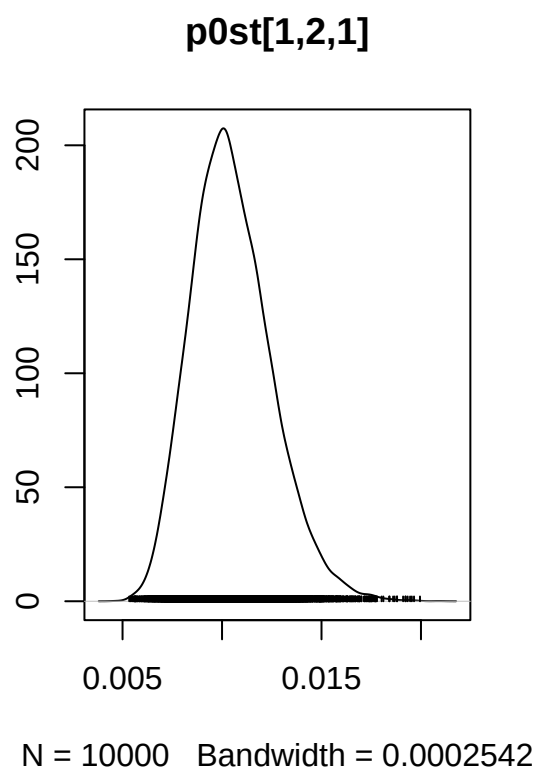
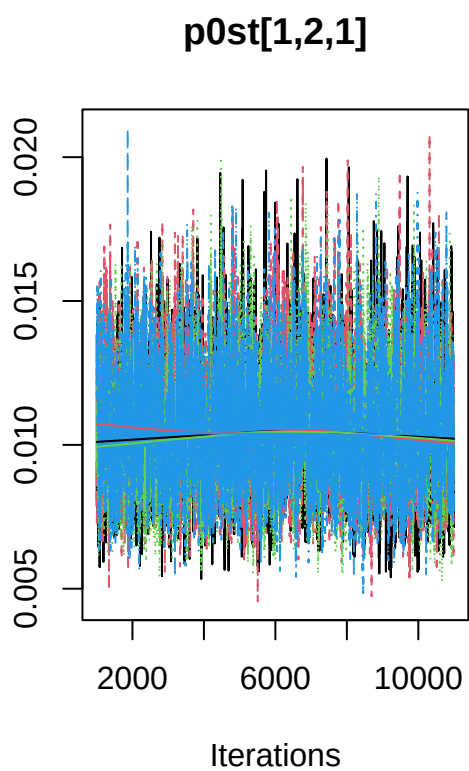


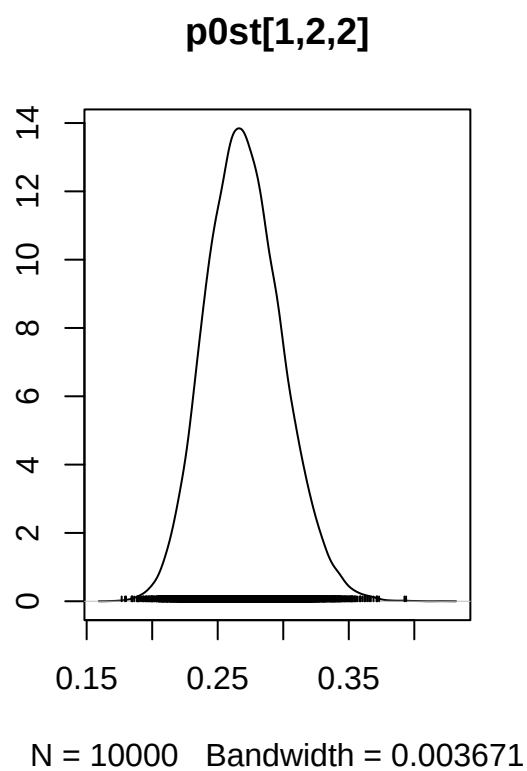
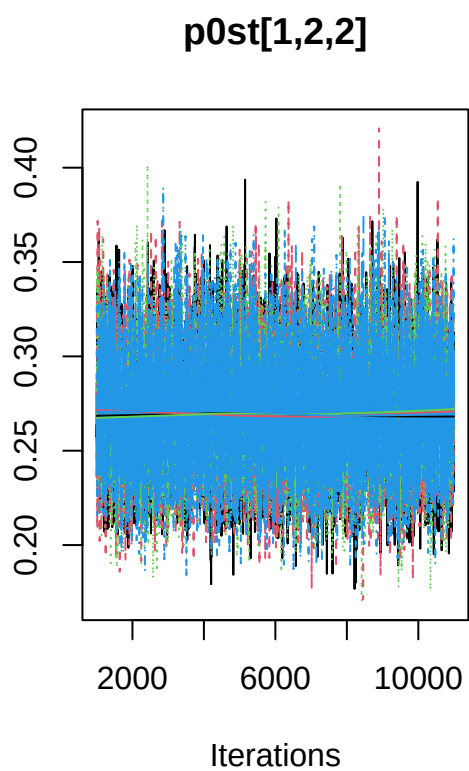
N = 10000 Bandwidth = 0.01155



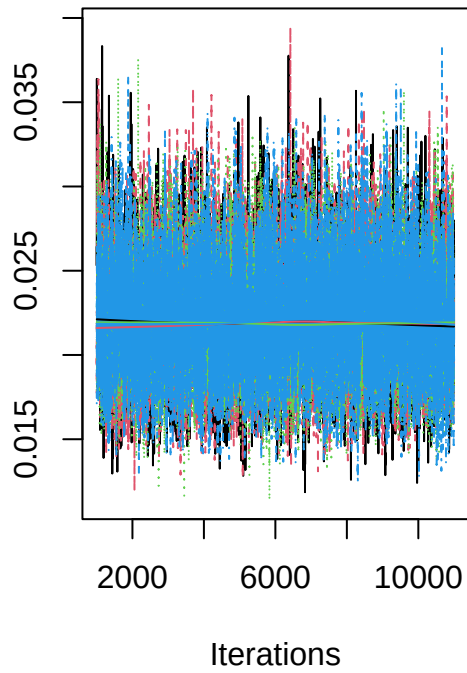




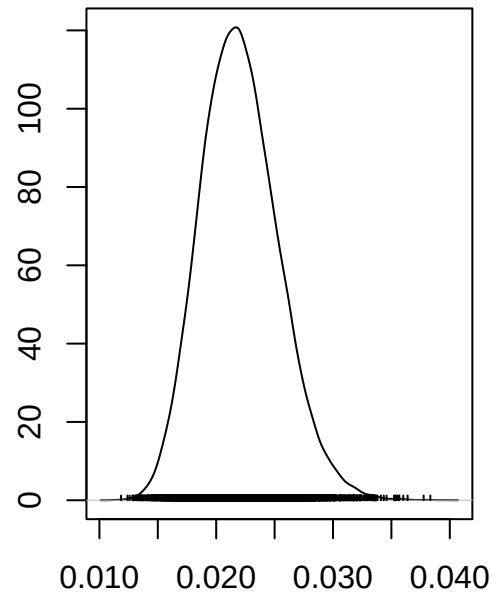




p0st[2,1,1]

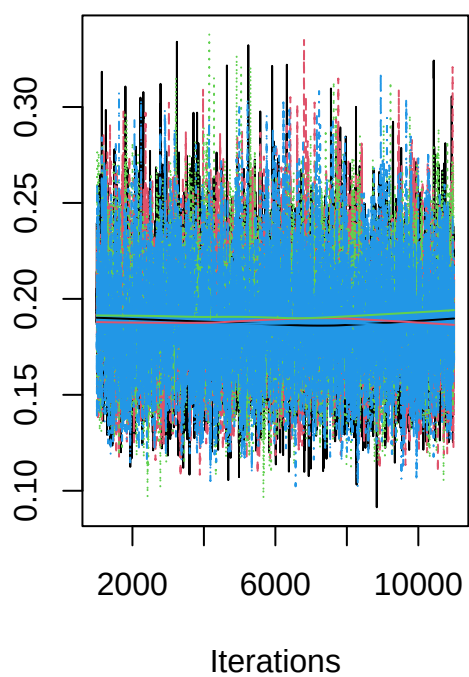


p0st[2,1,1]

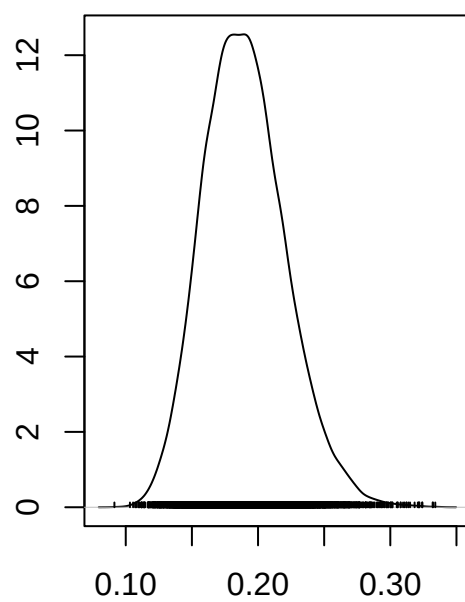


N = 10000 Bandwidth = 0.0004214

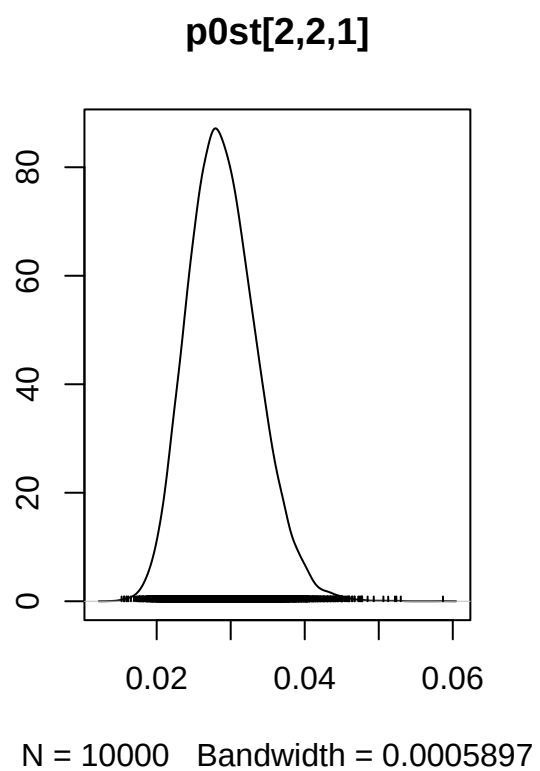
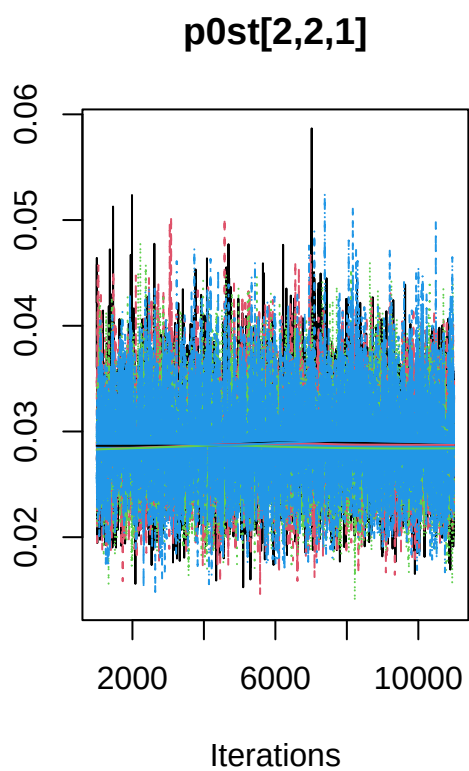
p0st[2,1,2]



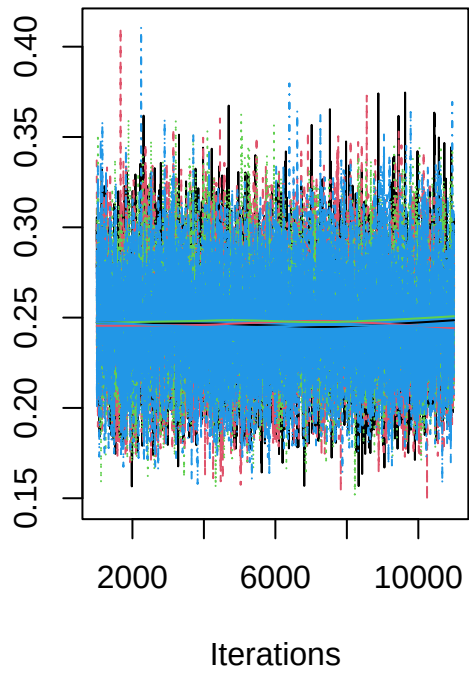
p0st[2,1,2]



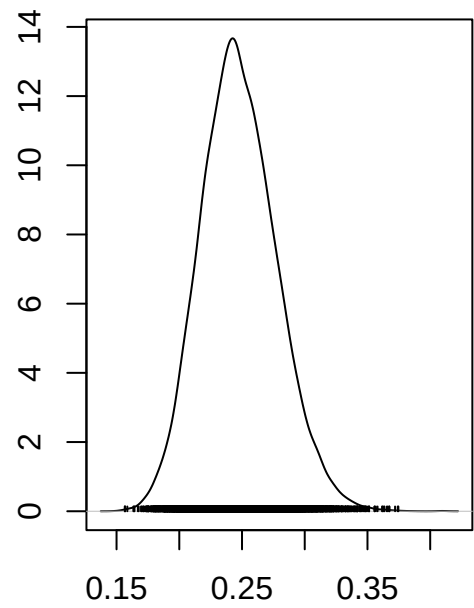
N = 10000 Bandwidth = 0.003948



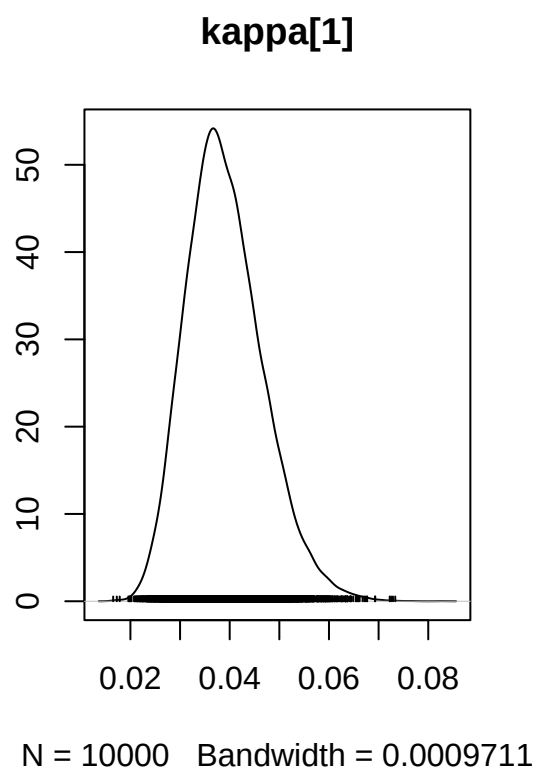
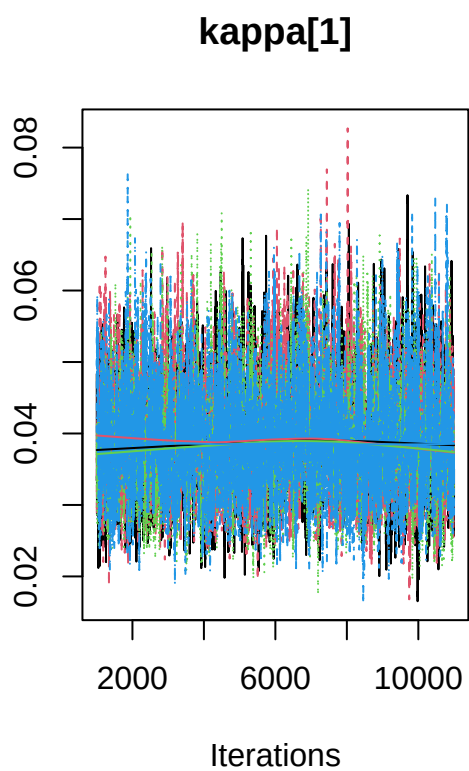
p0st[2,2,2]

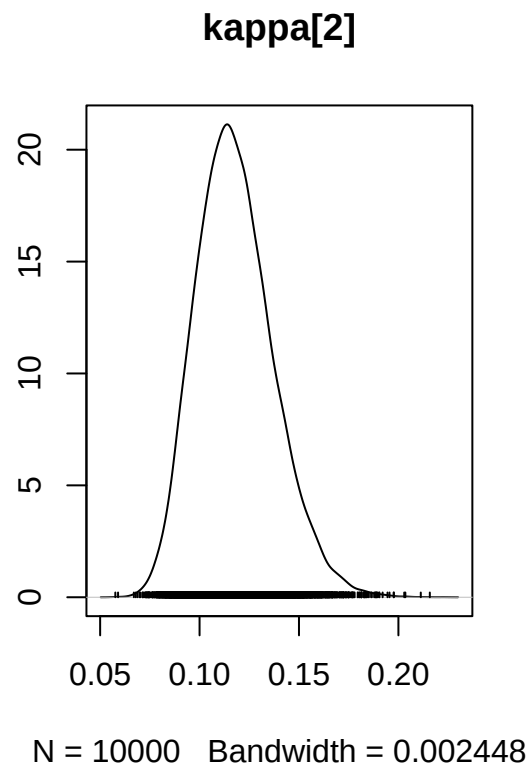
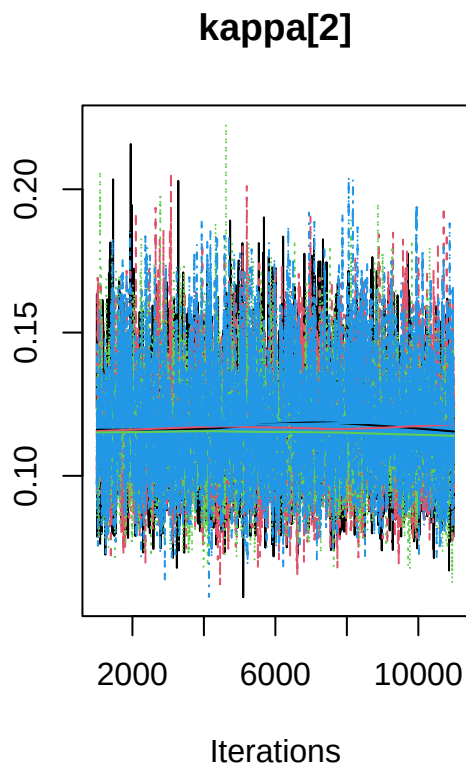


p0st[2,2,2]



N = 10000 Bandwidth = 0.003836





Activity Center Density

Now plot the density of inferred activity centers in the state space...

```
# Finally the density of activity centers in the study region
library(scrbook)
```

```
##
## Attaching package: 'scrbook'
```

```
## The following object is masked from 'package:raster':
##
## area
```

```
library(raster)
library(sf)
```

```
## Linking to GEOS 3.13.1, GDAL 3.11.0, PROJ 9.6.2; sf_use_s2() is TRUE
```

```
library(usmap)
library(prettypapr)
```

```
# Slightly modified plotting function
```

```

# See scrbook::SCRdensity() for the original
mySCRdensity <- function (obj, nx = 30, ny = 30, Xl = NULL, Xu = NULL, Yl = NULL,
  Yu = NULL, scalein = 100, scaleout = 100, col = "gray", ncolors = 10,
  whichguy = NULL) {
  Sxout <- obj$Sx
  Syout <- obj$Sy
  z <- obj$z
  niter <- nrow(z)
  if (is.null(Xl)) {
    Xl <- min(Sxout) * 0.999
    Xu <- max(Sxout) * 1.001
    Yl <- min(Syout) * 0.999
    Yu <- max(Syout) * 1.001
  }
  xg <- seq(Xl, Xu, , nx)
  yg <- seq(Yl, Yu, , ny)
  guy <- col(Sxout)
  Sxout <- cut(Sxout[z == 1], breaks = xg)
  Syout <- cut(Syout[z == 1], breaks = yg)
  if (is.null(whichguy)) {
    Dn <- table(Sxout, Syout)/niter
    area <- (yg[2] - yg[1]) * (xg[2] - xg[1]) * scalein
    Dn <- (Dn/area) * scaleout
  }
  else {
    Dn <- table(Sxout[guy == whichguy], Syout[guy == whichguy])/niter
  }
  Dn[Dn == 0] <- NA
  #cat("mean: ", mean(Dn), fill = TRUE)
  par(mar = c(3, 3, 3, 6))
  if (col == "gray") {
    cc <- seq(3, 17, , 10)/20
    cc <- rev(gray(cc))
  }
  else cc <- RColorBrewer::brewer.pal(9, "OrRd")[-1]
  image(xg, yg, Dn, col = cc)
  image.scale(Dn, col = cc)
  box()
  #return(list(grid = cbind(xg, yg), Dn = Dn))
}

gamap <- us_map(regions="counties", include="GA")
gamap <- st_transform(gamap, crs=4326) #Convert to WGS84
gamap_utm <- st_transform(gamap, crs=32617) #Convert to UTM

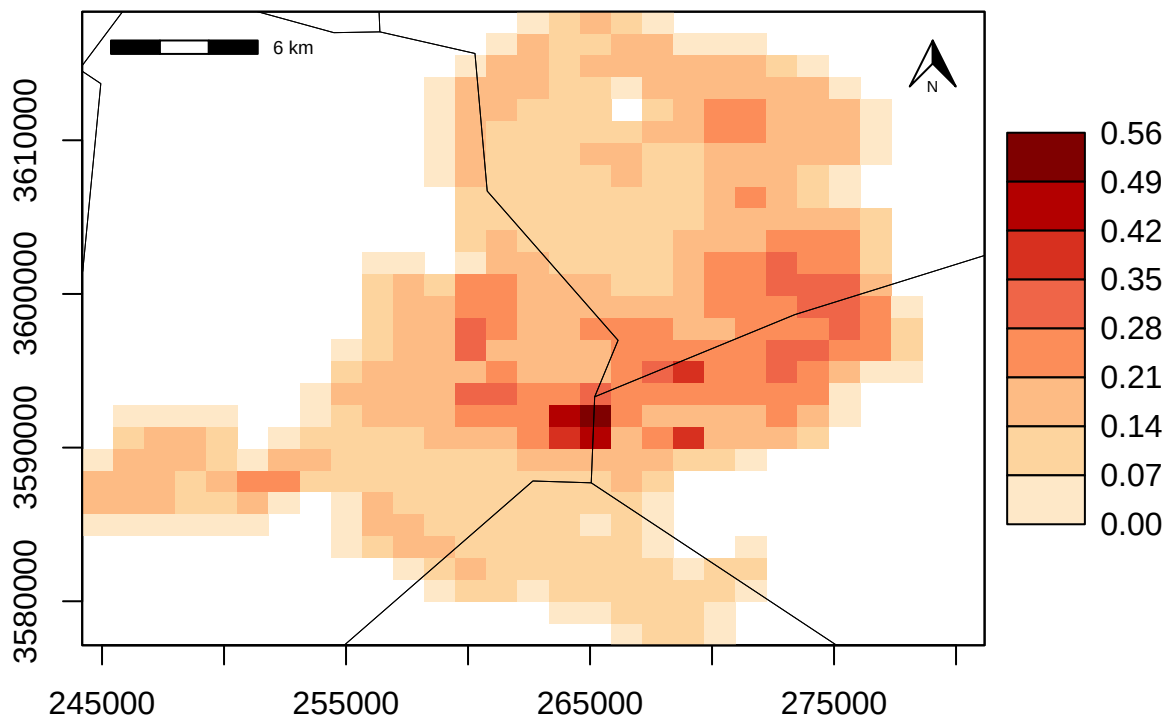
# Females
mat.out <- as.matrix(mc1)
M <- Mf
allS <- mat.out[,grep("sF", colnames(mat.out), fixed=TRUE)[1:(M*2)]]
Sx <- allS[,grep(",1", colnames(allS), fixed=TRUE)]
Sy <- allS[,grep(",2", colnames(allS), fixed=TRUE)]
z <- mat.out[,grep("zF", colnames(mat.out), fixed=TRUE)[1:M]]

```

```
obj <- list(Sx=Sx, Sy=Sy, z=z)

#Input coordinates are in m, output in ind/km^2 (i.e., 1e6=1000*1000)
mySCRdensity(obj, scalein=1, scaleout=1e6, col = "terrain",
             Xl=xlim0[1],
             Xu=xlim0[2],
             Yl=ylim0[1],
             Yu=ylim0[2],
             ncolors=10)
plot(st_geometry(gamap_utm), add=TRUE, lwd = 0.25)
addnortharrow("topright", scale = 0.4)
addscalebar(pos="topleft", htin=0.07, plotepsg =32617,
            widthhint = 0.2, label.cex = 0.6)
```

```
## Warning in CPL_crs_from_input(x): GDAL Message 1: +init=epsg:XXXX syntax is
## deprecated. It might return a CRS with a non-EPSG compliant axis order. Further
## messages of this type will be suppressed.
```



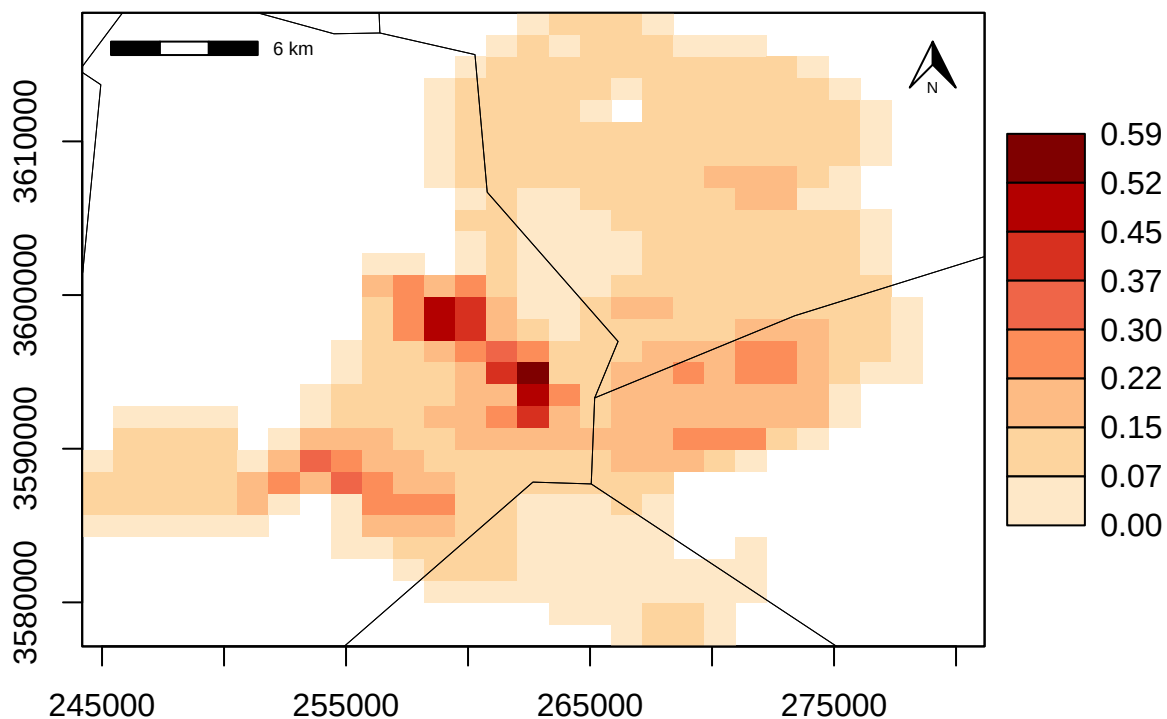
```
# Males
mat.out <- as.matrix(mc1)
M <- Mm
allS <- mat.out[,grep("sM[", colnames(mat.out), fixed=TRUE)[1:(M*2)]]
Sx <- allS[,grep(",1]", colnames(allS), fixed=TRUE)]
Sy <- allS[,grep(",2]", colnames(allS), fixed=TRUE)]
```

```

z <- mat.out[,grep("zM[", colnames(mat.out), fixed=TRUE)[1:M]]
obj <- list(Sx=Sx, Sy=Sy, z=z)

#Input coordinates are in m, output in ind/km^2 (i.e., 1e6=1000*1000)
mySCRdensity(obj, scalein=1, scaleout=1e6, col = "terrain",
             Xl=xlim0[1],
             Xu=xlim0[2],
             Yl=ylim0[1],
             Yu=ylim0[2],
             ncolors=10)
plot(st_geometry(gamap_utm), add=TRUE, lwd = 0.25)
addnortharrow("topright", scale = 0.4)
addscalebar(pos="topleft", htin=0.07, plotepsg =32617,
            widthhint = 0.2, label.cex = 0.6)

```



Save the workspace

Save the workspace and quit

```

# Save the entire workspace
save.image(file="Hooker_bothSexes.Rws")

```